
Python Grundlagen

Handout im Rahmen der Ausbildung DRV Bund

15.01.2026

1	Python-Grundlagen	3
1.1	Python-Installation und Entwicklungsumgebung einrichten	3
1.2	Erste Schritte: Python-Konsole und einfache Befehle	4
1.3	Variablen und grundlegende Datentypen	5
1.4	Namenskonventionen und Code-Formatierung	6
1.5	Zusammenfassung	8
2	Input/Output und Typumwandlung	9
2.1	Benutzereingaben und formatierte Ausgaben	9
2.2	Typumwandlung (Type Conversion)	11
2.3	Debugging-Grundlagen und Fehlerbehandlung	13
2.4	Zusammenfassung	15
3	Verzweigungen und Bedingungen	17
3.1	if, elif, else	17
3.2	Vergleichsoperatoren und Bedingungen	19
3.3	Code-Einrückung in Python	21
3.4	Logische Operatoren (and, or, not)	22
3.5	Verschachtelte Bedingungen	24
3.6	Zusammenfassung	25
4	Schleifen	27
4.1	While-Schleifen: Wiederholungen mit Bedingungen	27
4.2	For-Schleifen: Iteration über Bereiche	29
4.3	Die range()-Funktion	30
4.4	break und continue	32
4.5	Häufige Fehler bei Schleifen vermeiden	33
4.6	Praktische Beispiele	34
4.7	Zusammenfassung	36
5	Datenstrukturen	37
5.1	Listen erstellen und verwenden	37
5.2	Listen-Operationen: append, remove, insert, sort	39
5.3	Auf Listenelemente zugreifen (Indexierung)	41
5.4	Listen mit Schleifen durchlaufen	42
5.5	Dictionaries (Schlüssel-Wert-Paare)	43

5.6	Sets und Tuples (Überblick)	46
5.7	Zusammenfassung	47
6	Funktionen	49
6.1	Was sind Funktionen und warum brauchen wir sie?	49
6.2	Funktionen definieren und aufrufen	50
6.3	Parameter und Rückgabewerte	52
6.4	Lokale vs. globale Variablen	54
6.5	Best Practices: DRY-Prinzip (Don't Repeat Yourself)	55
6.6	Praktische Beispiele	57
6.7	Zusammenfassung	58
7	Python Standard Library	59
7.1	Wichtige Module: os, sys, pathlib, datetime	59
7.2	Module importieren und verwenden	64
7.3	Praktisches Beispiel	67
7.4	Zusammenfassung	68
8	Dateioperationen	69
8.1	Dateien lesen und schreiben	69
8.2	Context Manager (with-Statement)	71
8.3	CSV-Dateien verarbeiten	72
8.4	Praktisches Beispiel: Einfaches Logbuch-System	75
8.5	Zusammenfassung	76
8.6	Ausblick	76
9	Anhang	77
9.1	A. Grundlegende Kommandozeilen-Befehle	77
9.2	B. Python-Skripte ausführen	78
9.3	C. Offizielle Python-Dokumentation	81
9.4	D. Nützliche Tastenkombinationen	81
9.5	E. Häufige Fehler und Lösungen	82

Handout im Rahmen der Ausbildung DRV Bund

Willkommen zur deutschsprachigen Python-Dokumentation! Diese Dokumentation richtet sich an Einsteiger und vermittelt die grundlegenden Konzepte der Python-Programmierung.

Inhalt

Diese Dokumentation deckt folgende Themen ab:

1. **Python-Grundlagen** - Installation, Konsole, Variablen und Datentypen
2. **Input/Output und Typumwandlung** - Benutzereingaben und Datenkonvertierung
3. **Verzweigungen und Bedingungen** - if/elif/else und logische Operatoren
4. **Schleifen** - while, for, range() und Schleifenkontrolle
5. **Datenstrukturen** - Listen, Dictionaries, Sets und Tuples
6. **Funktionen** - Funktionsdefinition, Parameter und Best Practices
7. **Python Standard Library** - Wichtige Module und deren Verwendung
8. **Dateioperationen** - Dateien lesen, schreiben und verarbeiten

Aufbau der Kapitel

Jedes Kapitel ist so strukturiert, dass Erklärungen und praktische Codebeispiele im Verhältnis 50:50 stehen. Dies ermöglicht es Ihnen, die Konzepte direkt anhand von Beispielen zu verstehen und selbst auszuprobieren.

Voraussetzungen

- Python 3.8 oder höher
- Grundlegendes Verständnis für die Arbeit mit einem Computer
- Ein Texteditor oder eine IDE (z.B. VS Code, PyCharm)

Viel Erfolg beim Lernen!

Dieses Kapitel behandelt die grundlegenden Konzepte von Python, von der Installation bis zu den wichtigsten Datentypen und Namenskonventionen.

1.1 Python-Installation und Entwicklungsumgebung einrichten

Python ist eine interpretierte, objektorientierte Programmiersprache, die sich besonders gut für Einsteiger eignet. Um mit Python zu arbeiten, müssen Sie zunächst Python auf Ihrem System installieren.

1.1.1 Installation unter verschiedenen Betriebssystemen

Windows:

- Laden Sie Python von python.org herunter
- Führen Sie den Installer aus und aktivieren Sie “Add Python to PATH”
- Öffnen Sie die Eingabeaufforderung und prüfen Sie die Installation

Linux/Mac: Python ist oft vorinstalliert. Prüfen Sie die Version:

```
python3 --version
```

Falls nicht installiert, nutzen Sie den Paketmanager (z.B. apt, brew):

```
# Ubuntu/Debian
sudo apt update
sudo apt install python3

# macOS mit Homebrew
brew install python3
```

1.2 Erste Schritte: Python-Konsole und einfache Befehle

Die Python-Konsole (auch REPL - Read-Eval-Print-Loop genannt) ist ein interaktives Tool zum Testen von Code-Schnipseln.

1.2.1 Starten der Python-Konsole

Öffnen Sie ein Terminal und geben Sie ein:

```
python3
```

Sie sehen dann den Python-Prompt:

```
>>>
```

1.2.2 Einfache Berechnungen

Python kann direkt als Taschenrechner verwendet werden:

```
>>> 5 + 3
8

>>> 10 - 4
6

>>> 6 * 7
42

>>> 15 / 3
5.0

>>> 17 // 3  # Ganzzahldivision
5

>>> 17 % 3    # Modulo (Rest)
2

>>> 2 ** 8    # Potenz
256
```

1.2.3 Ausgabe mit print()

Die `print()`-Funktion gibt Text oder Werte auf der Konsole aus:

```
>>> print("Hallo Welt!")
Hallo Welt!

>>> print(42)
42
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
>>> print("Die Antwort ist:", 42)
Die Antwort ist: 42
```

1.3 Variablen und grundlegende Datentypen

Variablen sind Container für Werte. In Python müssen Sie Variablen nicht deklarieren - Sie weisen ihnen einfach einen Wert zu.

1.3.1 Variablen erstellen und verwenden

```
# Einfache Zuweisung
name = "Anna"
alter = 25
groesse = 1.75
ist_student = True

# Variablen ausgeben
print(name)          # Anna
print(alter)          # 25
print(groesse)        # 1.75
print(ist_student)    # True
```

1.3.2 Die wichtigsten Datentypen

Python hat mehrere grundlegende Datentypen:

Integer (int) - Ganze Zahlen:

```
anzahl = 10
temperatur = -5
jahr = 2024

# Operationen mit Integers
summe = anzahl + 100      # 110
produkt = anzahl * 5      # 50
```

Float - Fließkommazahlen (Dezimalzahlen):

```
preis = 19.99
pi = 3.14159
temperatur = -12.5

# Operationen mit Floats
total = preis * 2          # 39.98
umfang = 2 * pi * 5        # 31.4159
```

String (str) - Zeichenketten:

```
vorname = "Max"
nachname = 'Mustermann'
text = """Dies ist ein
mehrzeiliger Text"""

# String-Operationen
vollname = vorname + " " + nachname # Max Mustermann
gruß = "Hallo " * 3                 # Hallo Hallo Hallo
```

Boolean (bool) - Wahrheitswerte:

```
ist_aktiv = True
ist_abgeschlossen = False

# Boolean-Operationen
ergebnis = ist_aktiv and ist_abgeschlossen # False
ergebnis2 = ist_aktiv or ist_abgeschlossen # True
ergebnis3 = not ist_aktiv                  # False
```

1.3.3 Typ einer Variable prüfen

Mit `type()` können Sie den Datentyp einer Variable ermitteln:

```
zahl = 42
print(type(zahl))           # <class 'int'>

kommazahl = 3.14
print(type(kommazahl))      # <class 'float'>

text = "Python"
print(type(text))           # <class 'str'>

wahrheit = True
print(type(wahrheit))       # <class 'bool'>
```

1.4 Namenskonventionen und Code-Formatierung

Guter Code ist lesbarer Code. Python hat etablierte Konventionen für die Benennung von Variablen und die Formatierung von Code.

1.4.1 Regeln für Variablennamen

Erlaubt:

- Buchstaben (a-z, A-Z), Zahlen (0-9) und Unterstriche (`_`)
- Müssen mit einem Buchstaben oder Unterstrich beginnen
- Groß- und Kleinschreibung wird unterschieden

```
# Gültige Variablennamen
name = "Max"
alter_in_jahren = 25
_private_variable = 100
mitarbeiter2 = "Anna"
```

Nicht erlaubt:

```
# Ungültige Variablennamen (führen zu Fehlern)
# 2name = "Max"           # Beginnt mit Zahl
# mein-name = "Anna"      # Enthält Bindestrich
# for = 5                  # Reserviertes Schlüsselwort
```

1.4.2 Python Namenskonventionen (PEP 8)

PEP 8 ist der offizielle Style Guide für Python-Code:

```
# Variablen und Funktionen: snake_case (Kleinbuchstaben mit Unterstrichen)
benutzer_name = "Max"
anzahl_versuche = 3
max_wert = 100

# Konstanten: SCREAMING_SNAKE_CASE (Großbuchstaben mit Unterstrichen)
MAX_VERBINDUNGEN = 100
PI = 3.14159
DATENBANKNAME = "users"

# Klassen: PascalCase (jedes Wort beginnt mit Großbuchstaben)
# (wird später behandelt)
# class BankKonto:
#     pass
```

1.4.3 Code-Formatierung Best Practices

Gut formatierter Code ist leichter zu lesen und zu warten:

```
# Leerzeichen um Operatoren
x = 5 + 3           # Gut
# x=5+3            # Schlecht (schwer lesbar)

# Leerzeichen nach Kommas
werte = [1, 2, 3, 4, 5] # Gut
# werte = [1,2,3,4,5]   # Schlecht

# Aussagekräftige Variablennamen
benutzer_alter = 25    # Gut, selbsterklärend
# a = 25               # Schlecht, unklar

anzahl_fehler = 0      # Gut
# f = 0                # Schlecht

# Kommentare für komplexe Logik
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# Berechne den Rabatt basierend auf der Bestellmenge
if bestellmenge > 100:
    rabatt = 0.15
else:
    rabatt = 0.05
```

1.5 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- Wie Sie Python installieren und die Konsole verwenden
- Was Variablen sind und wie Sie diese erstellen
- Die grundlegenden Datentypen: `int`, `float`, `str`, `bool`
- Namenskonventionen und Code-Formatierung nach PEP 8

Im nächsten Kapitel lernen Sie, wie Sie Benutzereingaben verarbeiten und zwischen verschiedenen Datentypen konvertieren.

Input/Output und Typumwandlung

In diesem Kapitel lernen Sie, wie Sie mit Benutzereingaben arbeiten, Ausgaben formatieren und zwischen verschiedenen Datentypen konvertieren.

2.1 Benutzereingaben und formatierte Ausgaben

Interaktive Programme benötigen die Möglichkeit, Daten vom Benutzer entgegenzunehmen und Ergebnisse ansprechend auszugeben.

2.1.1 Die input()-Funktion

Mit `input()` können Sie Benutzereingaben entgegennehmen. Die Funktion gibt immer einen String zurück:

```
# Einfache Eingabe
name = input("Wie heißen Sie? ")
print("Hallo", name)

# Beispiel-Interaktion:
# Wie heißen Sie? Anna
# Hallo Anna
```

Wichtig: `input()` gibt immer einen String zurück, auch wenn der Benutzer eine Zahl eingibt:

```
eingabe = input("Geben Sie eine Zahl ein: ")
print(type(eingabe)) # <class 'str'>
# Auch bei Eingabe von "42" ist der Typ str!
```

2.1.2 Mehrere Eingaben verarbeiten

```
# Persönliche Daten abfragen
vorname = input("Vorname: ")
nachname = input("Nachname: ")
alter = input("Alter: ")

print("Willkommen,", vorname, nachname)
print("Sie sind", alter, "Jahre alt.")
```

2.1.3 Formatierte Ausgaben

Python bietet verschiedene Möglichkeiten, Ausgaben zu formatieren:

String-Konkatenation:

```
name = "Max"
alter = 30
print("Name: " + name + ", Alter: " + str(alter))
# Achtung: alter muss zu String konvertiert werden!
```

Format mit Komma-Separation:

```
name = "Anna"
punkte = 95
print("Spieler:", name, "- Punkte:", punkte)
# Ausgabe: Spieler: Anna - Punkte: 95
```

F-Strings (empfohlen ab Python 3.6):

```
name = "Lisa"
alter = 28
stadt = "Berlin"

print(f"Mein Name ist {name}, ich bin {alter} Jahre alt und wohne in {stadt}.")
# Ausgabe: Mein Name ist Lisa, ich bin 28 Jahre alt und wohne in Berlin.

# Mit Berechnungen:
preis = 19.99
menge = 3
print(f"Gesamtpreis: {preis * menge} Euro")
# Ausgabe: Gesamtpreis: 59.97 Euro
```

Format-Methode:

```
name = "Tom"
note = 1.7

print("Student: {}, Note: {}".format(name, note))
# Ausgabe: Student: Tom, Note: 1.7

# Mit benannten Platzhaltern:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
print("Name: {n}, Alter: {a}".format(n="Eva", a=25))  
# Ausgabe: Name: Eva, Alter: 25
```

2.1.4 Formatierung von Zahlen

F-Strings ermöglichen präzise Zahlenformatierung:

```
pi = 3.14159265  
  
# Auf 2 Dezimalstellen runden:  
print(f"Pi: {pi:.2f}")          # Pi: 3.14  
  
# Auf 4 Dezimalstellen:  
print(f"Pi: {pi:.4f}")          # Pi: 3.1416  
  
# Prozentangaben:  
anteil = 0.856  
print(f"Anteil: {anteil:.1%}")  # Anteil: 85.6%  
  
# Tausendertrennzeichen:  
betrag = 1234567.89  
print(f"Betrag: {betrag:,.2f} Euro")  # Betrag: 1,234,567.89 Euro
```

2.2 Typumwandlung (Type Conversion)

Oft müssen Daten von einem Typ in einen anderen konvertiert werden. Dies nennt man Type Casting oder Type Conversion.

2.2.1 Von String zu Zahl

Da `input()` immer Strings zurückgibt, ist die Konvertierung zu Zahlen besonders wichtig:

```
# String zu Integer:  
alter_str = "25"  
alter_int = int(alter_str)  
print(type(alter_int))  # <class 'int'>  
  
# Direkt bei der Eingabe konvertieren:  
alter = int(input("Ihr Alter: "))  
print("In 10 Jahren sind Sie", alter + 10, "Jahre alt.")  
  
# String zu Float:  
preis_str = "19.99"  
preis_float = float(preis_str)  
print(type(preis_float))  # <class 'float'>  
  
temperatur = float(input("Temperatur in Celsius: "))  
fahrenheit = temperatur * 9/5 + 32  
print(f"{temperatur}°C entspricht {fahrenheit:.1f}°F")
```

2.2.2 Von Zahl zu String

Um Zahlen mit Strings zu kombinieren, müssen sie oft zu Strings konvertiert werden:

```
alter = 30
text = "Ich bin " + str(alter) + " Jahre alt."
print(text)  # Ich bin 30 Jahre alt.

# Oder verwenden Sie f-Strings (einfacher):
print(f"Ich bin {alter} Jahre alt.")
```

2.2.3 Zwischen Integer und Float

```
# Float zu Integer (Nachkommastellen werden abgeschnitten):
kommazahl = 3.99
ganzzahl = int(kommazahl)
print(ganzzahl)  # 3 (nicht gerundet!)

# Integer zu Float:
zahl = 5
gleitkomma = float(zahl)
print(gleitkomma)  # 5.0
```

2.2.4 String zu Boolean

```
# Nicht-leere Strings sind True:
text = "Hallo"
wahrheit = bool(text)
print(wahrheit)  # True

# Leerer String ist False:
leer = ""
print(bool(leer))  # False

# Achtung: String "False" ist True!
falsch = "False"
print(bool(falsch))  # True (weil nicht leer)
```

2.2.5 Praktisches Beispiel: Einfacher Rechner

```
# Zwei Zahlen vom Benutzer einlesen:
zahl1 = float(input("Erste Zahl: "))
zahl2 = float(input("Zweite Zahl: "))

# Berechnungen durchführen:
summe = zahl1 + zahl2
differenz = zahl1 - zahl2
produkt = zahl1 * zahl2
quotient = zahl1 / zahl2
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# Ergebnisse formatiert ausgeben:
print(f"\nErgebnisse:")
print(f"{zahl1} + {zahl2} = {summe}")
print(f"{zahl1} - {zahl2} = {differenz}")
print(f"{zahl1} * {zahl2} = {produkt}")
print(f"{zahl1} / {zahl2} = {quotient:.2f}")
```

2.3 Debugging-Grundlagen und Fehlerbehandlung

Beim Arbeiten mit Benutzereingaben und Typkonvertierungen können Fehler auftreten. Es ist wichtig, diese zu verstehen und zu behandeln.

2.3.1 Häufige Fehler bei der Typkonvertierung

ValueError - Ungültige Konvertierung:

```
# Fehler: Kann "abc" nicht zu int konvertieren
# zahl = int("abc") # ValueError: invalid literal for int()

# Fehler: Kann "3.14" nicht direkt zu int konvertieren
# zahl = int("3.14") # ValueError: invalid literal for int()

# Lösung: Erst zu float, dann zu int:
zahl = int(float("3.14")) # Funktioniert: 3
```

ZeroDivisionError - Division durch Null:

```
# Fehler bei Division durch Null:
# ergebnis = 10 / 0 # ZeroDivisionError

# Besser: Prüfen vor der Division
nenner = 0
if nenner != 0:
    ergebnis = 10 / nenner
else:
    print("Fehler: Division durch Null nicht möglich!")
```

2.3.2 Einfaches Debugging mit print()

Die `print()`-Funktion ist das einfachste Debugging-Tool:

```
# Variablenwerte ausgeben:
x = 10
y = 20
print("Debug: x =", x, "y =", y)

# Typ einer Variable prüfen:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
eingabe = input("Zahl: ")
print("Typ der Eingabe:", type(eingabe))

# An verschiedenen Stellen Ausgaben einfügen:
print("Vor der Berechnung")
ergebnis = 5 * 10
print("Nach der Berechnung, Ergebnis:", ergebnis)
```

2.3.3 Try-Except für Fehlerbehandlung (Vorschau)

Ein kurzer Ausblick auf Fehlerbehandlung (wird später detailliert behandelt):

```
# Sichere Eingabe einer Zahl:
try:
    alter = int(input("Ihr Alter: "))
    print(f"Sie sind {alter} Jahre alt.")
except ValueError:
    print("Fehler: Bitte geben Sie eine gültige Zahl ein!")

# Beispiel-Interaktionen:
# Ihr Alter: 25
# Sie sind 25 Jahre alt.

# Ihr Alter: abc
# Fehler: Bitte geben Sie eine gültige Zahl ein!
```

2.3.4 Praktisches Beispiel: Robuste Benutzereingabe

```
# Programm zur Berechnung des BMI (Body Mass Index)

print("=== BMI-Rechner ===\n")

# Größe eingeben:
groesse_cm = float(input("Ihre Größe in cm: "))
groesse_m = groesse_cm / 100

# Gewicht eingeben:
gewicht = float(input("Ihr Gewicht in kg: "))

# BMI berechnen:
bmi = gewicht / (groesse_m ** 2)

# Ergebnis ausgeben:
print(f"\nIhr BMI: {bmi:.1f}")

if bmi < 18.5:
    print("Kategorie: Untergewicht")
elif bmi < 25:
    print("Kategorie: Normalgewicht")
elif bmi < 30:
    print("Kategorie: Übergewicht")
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
else:  
    print("Kategorie: Starkes Übergewicht")
```

2.4 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- Mit `input()` Benutzereingaben entgegenzunehmen
- Ausgaben mit f-Strings, `format()` und Komma-Separation zu formatieren
- Zwischen Datentypen zu konvertieren: `int()`, `float()`, `str()`, `bool()`
- Häufige Fehler bei der Typkonvertierung zu erkennen
- Einfache Debugging-Techniken mit `print()`

Im nächsten Kapitel lernen Sie Verzweigungen und Bedingungen kennen, um Ihren Code entscheidungsfähig zu machen.

Verzweigungen und Bedingungen

Verzweigungen ermöglichen es Ihrem Programm, Entscheidungen zu treffen und unterschiedliche Code-Pfade basierend auf Bedingungen auszuführen.

3.1 if, elif, else

Die grundlegenden Kontrollstrukturen für Verzweigungen in Python sind `if`, `elif` (else if) und `else`.

3.1.1 Die if-Anweisung

Eine `if`-Anweisung führt Code nur aus, wenn eine Bedingung wahr (True) ist:

```
alter = 18

if alter >= 18:
    print("Sie sind volljährig.")
    print("Sie dürfen wählen.")

# Ausgabe: Sie sind volljährig.
#           Sie dürfen wählen.
```

Wichtig: In Python wird die Einrückung verwendet, um Code-Blöcke zu definieren. Der eingetückte Code gehört zur `if`-Anweisung.

```
temperatur = 25

if temperatur > 20:
    print("Es ist warm draußen.")

print("Dieser Code wird immer ausgeführt.")
```

3.1.2 if-else

Mit `else` definieren Sie einen alternativen Code-Pfad, der ausgeführt wird, wenn die Bedingung falsch ist:

```
alter = 16

if alter >= 18:
    print("Sie sind volljährig.")
else:
    print("Sie sind noch minderjährig.")

# Ausgabe: Sie sind noch minderjährig.
```

Praktisches Beispiel:

```
temperatur = int(input("Temperatur in °C: "))

if temperatur >= 25:
    print("Es ist warm, tragen Sie leichte Kleidung.")
else:
    print("Es ist kühl, nehmen Sie eine Jacke mit.")
```

3.1.3 if-elif-else

Für mehrere Bedingungen verwenden Sie `elif` (else if):

```
note = 2

if note == 1:
    print("Sehr gut!")
elif note == 2:
    print("Gut!")
elif note == 3:
    print("Befriedigend.")
elif note == 4:
    print("Ausreichend.")
else:
    print("Nicht bestanden.")

# Ausgabe: Gut!
```

Mehrere Bedingungsgebiete:

```
punkte = 85

if punkte >= 90:
    note = 1
elif punkte >= 80:
    note = 2
elif punkte >= 70:
    note = 3
elif punkte >= 60:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
    note = 4
else:
    note = 5

print(f"Sie haben {punkte} Punkte erreicht: Note {note}")
# Ausgabe: Sie haben 85 Punkte erreicht: Note 2
```

3.1.4 Verschachtelte if-Anweisungen

Sie können if-Anweisungen ineinander verschachteln:

```
alter = 20
fuehrerschein = True

if alter >= 18:
    print("Sie sind volljährig.")
    if fuehrerschein:
        print("Sie dürfen Auto fahren.")
    else:
        print("Sie benötigen einen Führerschein.")
else:
    print("Sie sind zu jung zum Autofahren.")
```

3.2 Vergleichsoperatoren und Bedingungen

Vergleichsoperatoren werden verwendet, um Werte zu vergleichen und Bedingungen zu formulieren.

3.2.1 Die wichtigsten Vergleichsoperatoren

```
a = 10
b = 20

# Gleichheit
print(a == b)    # False (ist a gleich b?)

# Ungleichheit
print(a != b)    # True (ist a ungleich b?)

# Größer als
print(a > b)     # False (ist a größer als b?)

# Kleiner als
print(a < b)     # True (ist a kleiner als b?)

# Größer oder gleich
print(a >= 10)   # True (ist a größer oder gleich 10?)

# Kleiner oder gleich
print(b <= 20)   # True (ist b kleiner oder gleich 20?)
```

3.2.2 Vergleiche mit Strings

Strings können ebenfalls verglichen werden:

```
name = "Anna"

if name == "Anna":
    print("Hallo Anna!")

# Alphabetische Sortierung:
print("apple" < "banana")  # True (a kommt vor b)
print("Anna" < "anna")     # True (Großbuchstaben vor Kleinbuchstaben)

# Groß-/Kleinschreibung ignorieren:
eingabe = "PYTHON"
if eingabe.lower() == "python":
    print("Richtig!")  # Ausgabe: Richtig!
```

3.2.3 Mitgliedschaft prüfen (in, not in)

```
text = "Python ist großartig"

if "Python" in text:
    print("Python wurde gefunden!")  # Ausgabe: Python wurde gefunden!

if "Java" not in text:
    print("Java wurde nicht gefunden!")  # Ausgabe: Java wurde nicht_
gefunden!

# Mit Listen:
zahlen = [1, 2, 3, 4, 5]

if 3 in zahlen:
    print("3 ist in der Liste")  # Ausgabe: 3 ist in der Liste

if 10 not in zahlen:
    print("10 ist nicht in der Liste")  # Ausgabe: 10 ist nicht in der_
Liste
```


3.3 Code-Einrückung in Python

Python verwendet Einrückungen (Indentation), um Code-Blöcke zu definieren. Dies ist ein wichtiges Konzept!

3.3.1 Richtige Einrückung

```
alter = 25

if alter >= 18:
    print("Volljährig")           # Gehört zu if
    print("Darf wählen")         # Gehört zu if
print("Programm Ende")          # Gehört NICHT zu if

# Ausgabe:
# Volljährig
# Darf wählen
# Programm Ende
```

3.3.2 Falsche Einrückung führt zu Fehlern

```
# Fehler: Inkonsistente Einrückung
# if alter >= 18:
#     print("Zeile 1")
#         print("Zeile 2")  # Zu weit eingerückt!

# Fehler: Fehlende Einrückung
# if alter >= 18:
# print("Fehler")  # Muss eingerückt sein!
```

3.3.3 Empfehlung: 4 Leerzeichen

Der Python Style Guide (PEP 8) empfiehlt 4 Leerzeichen für jede Einrückungsebene:

```
if True:
    print("Ebene 1")
    if True:
        print("Ebene 2")
        if True:
            print("Ebene 3")
```

3.4 Logische Operatoren (and, or, not)

Logische Operatoren ermöglichen es, mehrere Bedingungen zu kombinieren.

3.4.1 Der and-Operator

Beide Bedingungen müssen wahr sein:

```
alter = 25
fuehrerschein = True

if alter >= 18 and fuehrerschein:
    print("Sie dürfen Auto fahren.")
# Ausgabe: Sie dürfen Auto fahren.

# Beispiel 2:
temperatur = 22
sonnig = True

if temperatur > 20 and sonnig:
    print("Perfektes Wetter für einen Ausflug!")
# Ausgabe: Perfektes Wetter für einen Ausflug!
```

Wahrheitstabelle für and:

```
print(True and True)    # True
print(True and False)   # False
print(False and True)   # False
print(False and False)  # False
```

3.4.2 Der or-Operator

Mindestens eine Bedingung muss wahr sein:

```
tag = "Samstag"

if tag == "Samstag" or tag == "Sonntag":
    print("Wochenende!")
# Ausgabe: Wochenende!

# Beispiel 2:
alter = 67

if alter < 18 or alter >= 65:
    print("Ermäßigter Eintritt")
# Ausgabe: Ermäßigter Eintritt
```

Wahrheitstabelle für or:

```
print(True or True)     # True
print(True or False)    # True
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
print(False or True)    # True
print(False or False)   # False
```

3.4.3 Der not-Operator

Kehrt einen Wahrheitswert um:

```
regen = False

if not regen:
    print("Kein Regen, gehen Sie spazieren!")
# Ausgabe: Kein Regen, gehen Sie spazieren!

# Beispiel 2:
ist_gesperrt = False

if not ist_gesperrt:
    print("Zugang erlaubt")
# Ausgabe: Zugang erlaubt
```

Wahrheitstabelle für not:

```
print(not True)    # False
print(not False)   # True
```

3.4.4 Komplexe Bedingungen

Sie können mehrere logische Operatoren kombinieren:

```
alter = 25
einkommen = 30000
kreditwuerdig = True

if alter >= 18 and einkommen > 20000 and kreditwuerdig:
    print("Kreditantrag genehmigt")
else:
    print("Kreditantrag abgelehnt")
# Ausgabe: Kreditantrag genehmigt

# Klammern für bessere Lesbarkeit:
x = 10
if (x > 5 and x < 15) or x == 0:
    print("Bedingung erfüllt")
# Ausgabe: Bedingung erfüllt
```

3.5 Verschachtelte Bedingungen

Manchmal müssen Entscheidungen von mehreren aufeinanderfolgenden Bedingungen abhängen.

3.5.1 Mehrfach verschachtelte if-Anweisungen

```
benutzername = "admin"
passwort = "geheim123"
aktiv = True

if benutzername == "admin":
    if passwort == "geheim123":
        if aktiv:
            print("Login erfolgreich")
        else:
            print("Account ist deaktiviert")
    else:
        print("Falsches Passwort")
else:
    print("Benutzername nicht gefunden")
```

3.5.2 Vereinfachung mit logischen Operatoren

Der obige Code kann vereinfacht werden:

```
benutzername = "admin"
passwort = "geheim123"
aktiv = True

if benutzername == "admin" and passwort == "geheim123" and aktiv:
    print("Login erfolgreich")
elif benutzername != "admin":
    print("Benutzername nicht gefunden")
elif passwort != "geheim123":
    print("Falsches Passwort")
else:
    print("Account ist deaktiviert")
```

3.5.3 Praktisches Beispiel: Ticketpreis berechnen

```
alter = int(input("Ihr Alter: "))
student = input("Sind Sie Student? (ja/nein): ").lower() == "ja"
wochentag = input("Welcher Tag? (Mo-So): ")

preis = 12.00 # Standardpreis

if alter < 6:
    preis = 0
    print("Kinder unter 6: kostenlos")
elif alter < 18:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
    preis = 8.00
    print("Jugendliche: ermäßigt")
elif alter >= 65:
    preis = 9.00
    print("Senioren: ermäßigt")
elif student:
    preis = 10.00
    print("Studenten: ermäßigt")

if wochentag in ["Montag", "Dienstag"]:
    preis = preis * 0.8
    print("Wochentag-Rabatt: 20%")

print(f"\nIhr Ticketpreis: {preis:.2f} Euro")
```

3.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- if, elif und else für Verzweigungen
- Vergleichsoperatoren: ==, !=, <, >, <=, >=
- Die Bedeutung der Code-Einrückung in Python
- Logische Operatoren: and, or, not
- Verschachtelte Bedingungen und deren Vereinfachung

Im nächsten Kapitel lernen Sie Schleifen kennen, um Code wiederholt auszuführen.

Schleifen ermöglichen es, Code mehrfach auszuführen, ohne ihn wiederholt schreiben zu müssen. Python bietet zwei Haupttypen von Schleifen: `while` und `for`.

4.1 While-Schleifen: Wiederholungen mit Bedingungen

Eine `while`-Schleife wiederholt einen Code-Block, solange eine Bedingung wahr ist.

4.1.1 Grundlegende while-Schleife

```
zaehler = 1

while zaehler <= 5:
    print(f"Durchlauf {zaehler}")
    zaehler += 1

print("Schleife beendet")

# Ausgabe:
# Durchlauf 1
# Durchlauf 2
# Durchlauf 3
# Durchlauf 4
# Durchlauf 5
# Schleife beendet
```

Wichtig: Die Bedingung muss irgendwann falsch werden, sonst entsteht eine Endlosschleife!

4.1.2 Endlosschleife (vermeiden!)

```
# VORSICHT: Diese Schleife läuft ewig!
# zaehler = 1
# while zaehler <= 5:
#     print(zaehler)
#     # Fehler: zaehler wird nie erhöht!
```

4.1.3 Praktische Beispiele

Benutzereingabe validieren:

```
passwort = ""

while passwort != "geheim":
    passwort = input("Bitte Passwort eingeben: ")
    if passwort != "geheim":
        print("Falsches Passwort, versuchen Sie es erneut.")

print("Zugang gewährt!")
```

Countdown:

```
countdown = 10

while countdown > 0:
    print(countdown)
    countdown -= 1

print("Start!")

# Ausgabe: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, Start!
```

Summe berechnen:

```
summe = 0
zahl = 1

while zahl <= 100:
    summe += zahl
    zahl += 1

print(f"Summe von 1 bis 100: {summe}")
# Ausgabe: Summe von 1 bis 100: 5050
```


4.2 For-Schleifen: Iteration über Bereiche

Eine `for`-Schleife iteriert über eine Sequenz (Liste, String, Bereich etc.).

4.2.1 For-Schleife mit `range()`

Die `range()`-Funktion erzeugt eine Zahlenfolge:

```
# Von 0 bis 4 (5 ist exklusiv):  
for i in range(5):  
    print(i)  
  
# Ausgabe: 0, 1, 2, 3, 4
```

Mit Start- und Endwert:

```
# Von 1 bis 10:  
for i in range(1, 11):  
    print(i, end=" ")  
  
print() # Zeilenumbruch  
# Ausgabe: 1 2 3 4 5 6 7 8 9 10
```

Mit Schrittweite:

```
# Gerade Zahlen von 0 bis 20:  
for i in range(0, 21, 2):  
    print(i, end=" ")  
  
# Ausgabe: 0 2 4 6 8 10 12 14 16 18 20
```

Rückwärts zählen:

```
for i in range(10, 0, -1):  
    print(i, end=" ")  
  
# Ausgabe: 10 9 8 7 6 5 4 3 2 1
```

4.2.2 Iteration über Strings

Sie können über jeden Buchstaben in einem String iterieren:

```
text = "Python"  
  
for buchstabe in text:  
    print(buchstabe)  
  
# Ausgabe:  
# P  
# y  
# t
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# h
# o
# n
```

Praktisches Beispiel:

```
wort = "Hallo"
vokal_anzahl = 0

for buchstabe in wort:
    if buchstabe.lower() in "aeiou":
        vokal_anzahl += 1

print(f"Das Wort '{wort}' enthält {vokal_anzahl} Vokale.")
# Ausgabe: Das Wort 'Hallo' enthält 2 Vokale.
```

4.2.3 Iteration über Listen

```
fruechte = ["Apfel", "Banane", "Orange", "Kiwi"]

for frucht in fruechte:
    print(f"Ich mag {frucht}")

# Ausgabe:
# Ich mag Apfel
# Ich mag Banane
# Ich mag Orange
# Ich mag Kiwi
```

Mit Index:

```
fruechte = ["Apfel", "Banane", "Orange"]

for index, frucht in enumerate(fruechte):
    print(f"{index + 1}. {frucht}")

# Ausgabe:
# 1. Apfel
# 2. Banane
# 3. Orange
```

4.3 Die range()-Funktion

Die `range()`-Funktion ist essenziell für `for`-Schleifen und hat drei Formen.

4.3.1 range(stop)

Erzeugt Zahlen von 0 bis stop-1:

```
for i in range(5):  
    print(i, end=" ")  
# Ausgabe: 0 1 2 3 4
```

4.3.2 range(start, stop)

Erzeugt Zahlen von start bis stop-1:

```
for i in range(3, 8):  
    print(i, end=" ")  
# Ausgabe: 3 4 5 6 7
```

4.3.3 range(start, stop, step)

Erzeugt Zahlen von start bis stop-1 mit Schrittweite step:

```
# Ungerade Zahlen:  
for i in range(1, 10, 2):  
    print(i, end=" ")  
# Ausgabe: 1 3 5 7 9  
  
print()  
  
# Rückwärts:  
for i in range(20, 10, -2):  
    print(i, end=" ")  
# Ausgabe: 20 18 16 14 12
```

4.3.4 Praktische Beispiele mit range()

Multiplikationstabelle:

```
zahl = 7  
  
print(f"Multiplikationstabelle für {zahl}:")  
for i in range(1, 11):  
    ergebnis = zahl * i  
    print(f"{zahl} x {i} = {ergebnis}")  
  
# Ausgabe:  
# Multiplikationstabelle für 7:  
# 7 x 1 = 7  
# 7 x 2 = 14  
# ...  
# 7 x 10 = 70
```

Liste mit Quadratzahlen:

```
quadrate = []

for i in range(1, 11):
    quadrate.append(i ** 2)

print(quadrate)
# Ausgabe: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

4.4 break und continue

Diese Schlüsselwörter ermöglichen es, den normalen Ablauf einer Schleife zu ändern.

4.4.1 break - Schleife vorzeitig beenden

break beendet die Schleife sofort:

```
for i in range(1, 11):
    if i == 5:
        break
    print(i)

print("Schleife wurde bei 5 abgebrochen")

# Ausgabe: 1, 2, 3, 4
# Schleife wurde bei 5 abgebrochen
```

Praktisches Beispiel:

```
# Zahl suchen:
zahlen = [10, 23, 45, 67, 89, 12, 34]
ziel = 67
gefunden = False

for zahl in zahlen:
    if zahl == ziel:
        print(f"Zahl {ziel} gefunden!")
        gefunden = True
        break

if not gefunden:
    print(f"Zahl {ziel} nicht gefunden.")

# Ausgabe: Zahl 67 gefunden!
```

4.4.2 continue - Aktuellen Durchlauf überspringen

continue überspringt den Rest des aktuellen Durchlaufs und geht zum nächsten:

```
for i in range(1, 11):
    if i % 2 == 0: # Gerade Zahlen überspringen
        continue
    print(i)

# Ausgabe: 1, 3, 5, 7, 9 (nur ungerade Zahlen)
```

Praktisches Beispiel:

```
# Negative Zahlen ignorieren:
zahlen = [5, -3, 8, -1, 12, -7, 15]

print("Positive Zahlen:")
for zahl in zahlen:
    if zahl < 0:
        continue
    print(zahl)

# Ausgabe: 5, 8, 12, 15
```

4.4.3 break vs. continue

```
print("Beispiel mit break:")
for i in range(5):
    if i == 3:
        break
    print(i)
# Ausgabe: 0, 1, 2 (stoppt bei 3)

print("\nBeispiel mit continue:")
for i in range(5):
    if i == 3:
        continue
    print(i)
# Ausgabe: 0, 1, 2, 4 (überspringt 3)
```

4.5 Häufige Fehler bei Schleifen vermeiden

4.5.1 Endlosschleifen

Achten Sie darauf, dass die Bedingung irgendwann falsch wird:

```
# FALSCH - Endlosschleife:
# i = 0
# while i < 10:
#     print(i)
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
#      # Fehler: i wird nie erhöht!

# RICHTIG:
i = 0
while i < 10:
    print(i)
    i += 1  # Bedingung wird irgendwann falsch
```

4.5.2 Off-by-One Fehler

```
# FALSCH - läuft nur 9 Mal:
# for i in range(1, 10):
#     print(i)  # Gibt 1-9 aus, nicht 1-10!

# RICHTIG - läuft 10 Mal:
for i in range(1, 11):
    print(i)  # Gibt 1-10 aus
```

4.5.3 Verschachtelte Schleifen

Achten Sie auf die Laufzeit bei verschachtelten Schleifen:

```
# Multiplikationstabelle:
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i} x {j} = {i*j}")
    print()  # Leerzeile nach jeder Reihe

# Diese Schleife läuft 5 x 5 = 25 Mal
```

Vorsicht bei vielen Verschachtelungen:

```
# Diese Schleife läuft 10 x 10 x 10 = 1000 Mal!
# for i in range(10):
#     for j in range(10):
#         for k in range(10):
#             # Rechenintensiver Code hier kann langsam werden
```

4.6 Praktische Beispiele

4.6.1 Fakultät berechnen

```
# n! = n x (n-1) x (n-2) x ... x 2 x 1
n = 5
fakultaet = 1

for i in range(1, n + 1):
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
fakultaet *= i

print(f"{n}! = {fakultaet}")
# Ausgabe: 5! = 120
```

4.6.2 Primzahlen finden

```
# Alle Primzahlen bis 50 finden:
print("Primzahlen bis 50:")

for zahl in range(2, 51):
    ist_primzahl = True

    for teiler in range(2, int(zahl ** 0.5) + 1):
        if zahl % teiler == 0:
            ist_primzahl = False
            break

    if ist_primzahl:
        print(zahl, end=" ")

# Ausgabe: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

4.6.3 Einfaches Zahlenratespiel

```
import random

ziel = random.randint(1, 100)
versuche = 0
max_versuche = 10

print("Zahlenratespiel: Errate die Zahl zwischen 1 und 100!")

while versuche < max_versuche:
    rate = int(input(f"Versuch {versuche + 1}/{max_versuche}: "))
    versuche += 1

    if rate == ziel:
        print(f"Richtig! Sie haben die Zahl in {versuche} Versuchen erraten.")
        break
    elif rate < ziel:
        print("Zu niedrig!")
    else:
        print("Zu hoch!")
else:
    print(f"Leider verloren. Die Zahl war {ziel}.")
```

4.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- `while`-Schleifen für bedingte Wiederholungen
- `for`-Schleifen für Iteration über Sequenzen
- Die `range()`-Funktion in ihren verschiedenen Formen
- `break` zum vorzeitigen Beenden von Schleifen
- `continue` zum Überspringen von Durchläufen
- Häufige Fehler und wie man sie vermeidet

Im nächsten Kapitel lernen Sie Datenstrukturen wie Listen, Dictionaries, Sets und Tuples kennen.

Datenstrukturen ermöglichen es, mehrere Werte in einer einzigen Variable zu speichern und effizient zu verwalten. Python bietet vier grundlegende Datenstrukturen: Listen, Dictionaries, Sets und Tuples.

5.1 Listen erstellen und verwenden

Listen sind geordnete, veränderbare Sammlungen von Elementen. Sie können Elemente unterschiedlicher Datentypen enthalten.

5.1.1 Listen erstellen

```
# Leere Liste:
leere_liste = []

# Liste mit Zahlen:
zahlen = [1, 2, 3, 4, 5]

# Liste mit Strings:
fruechte = ["Apfel", "Banane", "Orange"]

# Gemischte Liste:
gemischt = [1, "Text", 3.14, True]

print(zahlen)      # [1, 2, 3, 4, 5]
print(fruechte)    # ['Apfel', 'Banane', 'Orange']
```

5.1.2 Auf Listenelemente zugreifen (Indexierung)

Listen verwenden null-basierte Indexierung:

```
fruechte = ["Apfel", "Banane", "Orange", "Kiwi"]

# Erstes Element (Index 0):
print(fruechte[0])    # Apfel

# Zweites Element:
print(fruechte[1])    # Banane

# Letztes Element:
print(fruechte[-1])   # Kiwi

# Vorletztes Element:
print(fruechte[-2])   # Orange
```

5.1.3 Slicing - Teillisten extrahieren

```
zahlen = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Von Index 2 bis 5 (exklusive):
print(zahlen[2:5])    # [2, 3, 4]

# Von Anfang bis Index 4:
print(zahlen[:4])     # [0, 1, 2, 3]

# Von Index 6 bis Ende:
print(zahlen[6:])     # [6, 7, 8, 9]

# Jedes zweite Element:
print(zahlen[::2])    # [0, 2, 4, 6, 8]

# Liste umkehren:
print(zahlen[::-1])   # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

5.2 Listen-Operationen: append, remove, insert, sort

Listen sind veränderbar (mutable), d.h. Sie können Elemente hinzufügen, entfernen und ändern.

5.2.1 Elemente hinzufügen

append() - Am Ende anfügen:

```
fruechte = ["Apfel", "Banane"]
fruechte.append("Orange")
print(fruechte)  # ['Apfel', 'Banane', 'Orange']

# Mehrere Elemente hinzufügen:
zahlen = [1, 2, 3]
zahlen.append(4)
zahlen.append(5)
print(zahlen)  # [1, 2, 3, 4, 5]
```

insert() - An bestimmter Position einfügen:

```
fruechte = ["Apfel", "Orange"]
fruechte.insert(1, "Banane")  # An Index 1 einfügen
print(fruechte)  # ['Apfel', 'Banane', 'Orange']

# Am Anfang einfügen:
zahlen = [2, 3, 4]
zahlen.insert(0, 1)
print(zahlen)  # [1, 2, 3, 4]
```

extend() - Mehrere Elemente anfügen:

```
liste1 = [1, 2, 3]
liste2 = [4, 5, 6]
liste1.extend(liste2)
print(liste1)  # [1, 2, 3, 4, 5, 6]

# Alternative: + Operator
liste_a = [1, 2]
liste_b = [3, 4]
liste_c = liste_a + liste_b
print(liste_c)  # [1, 2, 3, 4]
```

5.2.2 Elemente entfernen

remove() - Erstes Vorkommen entfernen:

```
fruechte = ["Apfel", "Banane", "Orange", "Banane"]
fruechte.remove("Banane")
print(fruechte)  # ['Apfel', 'Orange', 'Banane']
```

pop() - Element an Index entfernen (und zurückgeben):

```
zahlen = [10, 20, 30, 40]

# Letztes Element entfernen:
letztes = zahlen.pop()
print(letztes)    # 40
print(zahlen)     # [10, 20, 30]

# Element an Index 1 entfernen:
element = zahlen.pop(1)
print(element)    # 20
print(zahlen)     # [10, 30]
```

clear() - Alle Elemente entfernen:

```
liste = [1, 2, 3, 4, 5]
liste.clear()
print(liste)     # []
```

5.2.3 Listen sortieren und umkehren

sort() - Liste sortieren (in-place):

```
zahlen = [3, 1, 4, 1, 5, 9, 2]
zahlen.sort()
print(zahlen)    # [1, 1, 2, 3, 4, 5, 9]

# Absteigend sortieren:
zahlen.sort(reverse=True)
print(zahlen)    # [9, 5, 4, 3, 2, 1, 1]

# Strings alphabetisch sortieren:
namen = ["Zoe", "Anna", "Max", "Ben"]
namen.sort()
print(namen)     # ['Anna', 'Ben', 'Max', 'Zoe']
```

sorted() - Neue sortierte Liste erstellen:

```
original = [3, 1, 4, 1, 5]
sortiert = sorted(original)
print(original)  # [3, 1, 4, 1, 5] (unverändert)
print(sortiert)  # [1, 1, 3, 4, 5]
```

reverse() - Liste umkehren:

```
zahlen = [1, 2, 3, 4, 5]
zahlen.reverse()
print(zahlen)    # [5, 4, 3, 2, 1]
```

5.2.4 Weitere nützliche Listen-Methoden

```
fruechte = ["Apfel", "Banane", "Orange", "Banane"]

# Länge der Liste:
print(len(fruechte))    # 4

# Anzahl eines Elements:
print(fruechte.count("Banane"))    # 2

# Index eines Elements finden:
print(fruechte.index("Orange"))    # 2

# Prüfen, ob Element vorhanden:
print("Apfel" in fruechte)    # True
print("Kiwi" in fruechte)    # False
```

5.3 Auf Listenelemente zugreifen (Indexierung)

Wir haben bereits Indexierung kennengelernt. Hier weitere wichtige Konzepte:

5.3.1 Negative Indizes

```
zahlen = [10, 20, 30, 40, 50]

print(zahlen[-1])    # 50 (letztes Element)
print(zahlen[-2])    # 40 (vorletztes Element)
print(zahlen[-5])    # 10 (erstes Element)
```

5.3.2 Listen verschachteln

```
# Matrix (2D-Liste):
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Auf Elemente zugreifen:
print(matrix[0])    # [1, 2, 3] (erste Zeile)
print(matrix[0][0])    # 1
print(matrix[1][2])    # 6
print(matrix[2][1])    # 8

# Über Matrix iterieren:
for zeile in matrix:
    for element in zeile:
        print(element, end=" ")
    print()
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# Ausgabe:  
# 1 2 3  
# 4 5 6  
# 7 8 9
```

5.3.3 List Comprehensions

Eine kompakte Methode, Listen zu erstellen:

```
# Traditionell:  
quadrate = []  
for x in range(10):  
    quadrate.append(x ** 2)  
print(quadrate)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]  
  
# Mit List Comprehension:  
quadrate = [x ** 2 for x in range(10)]  
print(quadrate)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]  
  
# Mit Bedingung:  
gerade = [x for x in range(20) if x % 2 == 0]  
print(gerade)  # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

5.4 Listen mit Schleifen durchlaufen

5.4.1 Mit for-Schleife

```
fruechte = ["Apfel", "Banane", "Orange"]  
  
# Über Elemente iterieren:  
for frucht in fruechte:  
    print(f"Ich mag {frucht}")  
  
# Mit Index:  
for i in range(len(fruechte)):  
    print(f"{i}: {fruechte[i]}")  
  
# Mit enumerate (empfohlen):  
for index, frucht in enumerate(fruechte):  
    print(f"{index}: {frucht}")
```

5.4.2 Liste filtern und transformieren

```
zahlen = [1, -5, 3, -2, 8, -7, 4]

# Nur positive Zahlen:
positive = []
for zahl in zahlen:
    if zahl > 0:
        positive.append(zahl)
print(positive)  # [1, 3, 8, 4]

# Oder mit List Comprehension:
positive = [zahl for zahl in zahlen if zahl > 0]
print(positive)  # [1, 3, 8, 4]

# Alle Werte verdoppeln:
verdoppelt = [zahl * 2 for zahl in zahlen]
print(verdoppelt)  # [2, -10, 6, -4, 16, -14, 8]
```

5.5 Dictionaries (Schlüssel-Wert-Paare)

Dictionaries speichern Daten als Schlüssel-Wert-Paare. Sie sind ungeordnet (ab Python 3.7 Einfügereihenfolge erhalten) und veränderbar.

5.5.1 Dictionaries erstellen

```
# Leeres Dictionary:
leeres_dict = {}

# Dictionary mit Daten:
person = {
    "name": "Anna",
    "alter": 25,
    "stadt": "Berlin"
}

# Alternative Syntax:
produkt = dict(name="Laptop", preis=999, lagerbestand=50)

print(person)  # {'name': 'Anna', 'alter': 25, 'stadt': 'Berlin'}
print(produkt)  # {'name': 'Laptop', 'preis': 999, 'lagerbestand': 50}
```

5.5.2 Auf Werte zugreifen

```
person = {
    "name": "Max",
    "alter": 30,
    "beruf": "Programmierer"
}

# Mit Schlüssel:
print(person["name"])    # Max
print(person["alter"])   # 30

# Mit get() (sicherer):
print(person.get("beruf"))      # Programmierer
print(person.get("telefon"))    # None (kein Fehler)
print(person.get("telefon", "Nicht vorhanden")) # Nicht vorhanden
```

5.5.3 Werte ändern und hinzufügen

```
person = {"name": "Lisa", "alter": 28}

# Wert ändern:
person["alter"] = 29
print(person)    # {'name': 'Lisa', 'alter': 29}

# Neues Schlüssel-Wert-Paar hinzufügen:
person["stadt"] = "München"
print(person)    # {'name': 'Lisa', 'alter': 29, 'stadt': 'München'}

# Mehrere Paare hinzufügen mit update():
person.update({"beruf": "Ärztin", "hobby": "Lesen"})
print(person)
# {'name': 'Lisa', 'alter': 29, 'stadt': 'München', 'beruf': 'Ärztin',
#  'hobby': 'Lesen'}
```

5.5.4 Elemente entfernen

```
person = {"name": "Tom", "alter": 35, "stadt": "Hamburg"}

# Mit pop():
alter = person.pop("alter")
print(alter)    # 35
print(person)   # {'name': 'Tom', 'stadt': 'Hamburg'}

# Mit del:
del person["stadt"]
print(person)   # {'name': 'Tom'}

# Alle Elemente entfernen:
person.clear()
print(person)   # {}
```


5.5.5 Über Dictionaries iterieren

```
noten = {"Anna": 1, "Max": 2, "Lisa": 1, "Tom": 3}

# Über Schlüssel iterieren:
for name in noten:
    print(name)

# Über Schlüssel (explizit):
for name in noten.keys():
    print(name, noten[name])

# Über Werte:
for note in noten.values():
    print(note)

# Über Schlüssel und Werte:
for name, note in noten.items():
    print(f"{name} hat Note {note}")

# Ausgabe:
# Anna hat Note 1
# Max hat Note 2
# Lisa hat Note 1
# Tom hat Note 3
```

5.5.6 Nützliche Dictionary-Methoden

```
person = {"name": "Eva", "alter": 27, "stadt": "Köln"}

# Alle Schlüssel:
print(person.keys())    # dict_keys(['name', 'alter', 'stadt'])

# Alle Werte:
print(person.values())  # dict_values(['Eva', 27, 'Köln'])

# Alle Paare:
print(person.items())   # dict_items([('name', 'Eva'), ('alter', 27), ('stadt', 'Köln')])

# Prüfen, ob Schlüssel existiert:
print("name" in person)    # True
print("telefon" in person) # False

# Anzahl der Einträge:
print(len(person))        # 3
```

5.6 Sets und Tuples (Überblick)

5.6.1 Sets - Ungeordnete Mengen ohne Duplikate

Sets sind ungeordnete Sammlungen eindeutiger Elemente:

```
# Set erstellen:
fruechte = {"Apfel", "Banane", "Orange"}
zahlen = set([1, 2, 3, 2, 1]) # Duplikate werden entfernt
print(zahlen) # {1, 2, 3}

# Element hinzufügen:
fruechte.add("Kiwi")
print(fruechte) # {'Apfel', 'Banane', 'Orange', 'Kiwi'}

# Element entfernen:
fruechte.remove("Banane")
print(fruechte) # {'Apfel', 'Orange', 'Kiwi'}

# Mengenoperationen:
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

print(set_a | set_b) # Vereinigung: {1, 2, 3, 4, 5, 6}
print(set_a & set_b) # Schnittmenge: {3, 4}
print(set_a - set_b) # Differenz: {1, 2}
```

Praktisches Beispiel - Duplikate entfernen:

```
zahlen_mit_duplikaten = [1, 2, 2, 3, 4, 4, 4, 5]
eindeutige_zahlen = list(set(zahlen_mit_duplikaten))
print(eindeutige_zahlen) # [1, 2, 3, 4, 5] (Reihenfolge kann variieren)
```

5.6.2 Tuples - Unveränderbare Sequenzen

Tuples sind wie Listen, aber unveränderbar (immutable):

```
# Tuple erstellen:
koordinaten = (10, 20)
person = ("Anna", 25, "Berlin")

# Auf Elemente zugreifen:
print(koordinaten[0]) # 10
print(person[1])      # 25

# Tuple Unpacking:
x, y = koordinaten
print(f"x={x}, y={y}") # x=10, y=20

name, alter, stadt = person
print(f"{name} ist {alter} Jahre alt") # Anna ist 25 Jahre alt
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# Tuples sind unveränderbar:  
# koordinaten[0] = 15 # Fehler: TypeError
```

Wann Tuples verwenden?

```
# Für unveränderliche Daten:  
rgb_farbe = (255, 128, 0)  
  
# Als Dictionary-Schlüssel (Listen gehen nicht):  
koordinaten_dict = {  
    (0, 0): "Ursprung",  
    (1, 0): "X-Achse",  
    (0, 1): "Y-Achse"  
}  
  
# Mehrere Werte aus Funktion zurückgeben:  
def min_max(zahlen):  
    return min(zahlen), max(zahlen)  
  
minimum, maximum = min_max([1, 5, 3, 9, 2])  
print(f"Min: {minimum}, Max: {maximum}") # Min: 1, Max: 9
```

5.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- **Listen:** Geordnete, veränderbare Sequenzen mit Methoden wie `append()`, `remove()`, `insert()`, `sort()`
- **Indexierung und Slicing:** Zugriff auf Elemente mit `[]`, negative Indizes, Slicing mit `[start:stop:step]`
- **Dictionaries:** Schlüssel-Wert-Paare mit Methoden wie `get()`, `keys()`, `values()`, `items()`
- **Sets:** Ungeordnete Mengen ohne Duplikate
- **Tuples:** Unveränderbare Sequenzen

Im nächsten Kapitel lernen Sie, wie Sie Funktionen definieren und verwenden, um Code wiederverwendbar zu machen.

Funktionen sind wiederverwendbare Code-Blöcke, die eine bestimmte Aufgabe erfüllen. Sie machen Code übersichtlicher, wartbarer und vermeiden Wiederholungen.

6.1 Was sind Funktionen und warum brauchen wir sie?

Funktionen gruppieren zusammengehörigen Code unter einem Namen und ermöglichen es, diesen Code mehrfach aufzurufen, ohne ihn zu wiederholen.

6.1.1 Vorteile von Funktionen

Ohne Funktionen (Code-Wiederholung):

```
# Berechnung 1:
radius1 = 5
flaeche1 = 3.14159 * radius1 ** 2
print(f"Fläche: {flaeche1}")

# Berechnung 2:
radius2 = 10
flaeche2 = 3.14159 * radius2 ** 2
print(f"Fläche: {flaeche2}")

# Berechnung 3:
radius3 = 7
flaeche3 = 3.14159 * radius3 ** 2
print(f"Fläche: {flaeche3}")
```

Mit Funktion (DRY - Don't Repeat Yourself):

```
def kreis_flaeche(radius):  
    """Berechnet die Fläche eines Kreises."""  
    return 3.14159 * radius ** 2  
  
# Jetzt einfach mehrfach verwenden:  
print(f"Fläche: {kreis_flaeche(5)}")  
print(f"Fläche: {kreis_flaeche(10)}")  
print(f"Fläche: {kreis_flaeche(7)}")
```

6.1.2 Vorteile auf einen Blick

- **Wiederverwendbarkeit:** Code einmal schreiben, mehrfach nutzen
- **Lesbarkeit:** Aussagekräftige Namen dokumentieren, was der Code tut
- **Wartbarkeit:** Änderungen nur an einer Stelle nötig
- **Testbarkeit:** Funktionen können isoliert getestet werden

6.2 Funktionen definieren und aufrufen

Eine Funktion wird mit dem Schlüsselwort `def` definiert.

6.2.1 Einfache Funktion ohne Parameter

```
def begruessung():  
    """Gibt eine Begrüßung aus."""  
    print("Hallo und willkommen!")  
    print("Schön, dass Sie da sind.")  
  
# Funktion aufrufen:  
begruessung()  
# Ausgabe:  
# Hallo und willkommen!  
# Schön, dass Sie da sind.  
  
# Funktion mehrfach aufrufen:  
begruessung()  
begruessung()
```

6.2.2 Funktion mit einem Parameter

```
def begruesse(name):  
    """Begrüßt eine Person mit ihrem Namen."""  
    print(f"Hallo {name}!")  
    print(f"Schön, Sie zu sehen, {name}.")  
  
# Funktionsaufrufe:  
begruesse("Anna")
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
begruesse("Max")
begruesse("Lisa")

# Ausgabe:
# Hallo Anna!
# Schön, Sie zu sehen, Anna.
# Hallo Max!
# Schön, Sie zu sehen, Max.
# ...
```

6.2.3 Funktion mit mehreren Parametern

```
def addiere(a, b):
    """Addiert zwei Zahlen und gibt das Ergebnis aus."""
    summe = a + b
    print(f"{a} + {b} = {summe}")

addiere(5, 3)          # 5 + 3 = 8
addiere(10, 20)        # 10 + 20 = 30
addiere(7, -2)         # 7 + -2 = 5
```

6.2.4 Funktion mit return-Wert

Funktionen können Werte zurückgeben, die weiterverwendet werden können:

```
def multipliziere(a, b):
    """Multipliziert zwei Zahlen und gibt das Ergebnis zurück."""
    return a * b

# Rückgabewert verwenden:
ergebnis = multipliziere(4, 5)
print(ergebnis)  # 20

# Direkt in Ausdrücken verwenden:
gesamt = multipliziere(3, 4) + multipliziere(2, 5)
print(gesamt)    # 12 + 10 = 22

# In Bedingungen verwenden:
if multipliziere(2, 3) > 5:
    print("Größer als 5")  # Wird ausgegeben
```

6.3 Parameter und Rückgabewerte

Parameter ermöglichen es, Daten an Funktionen zu übergeben. Rückgabewerte ermöglichen es Funktionen, Ergebnisse zurückzugeben.

6.3.1 Positionsparameter

Die Reihenfolge der Argumente muss der Reihenfolge der Parameter entsprechen:

```
def vorstellen(name, alter, stadt):  
    """Stellt eine Person vor."""  
    print(f"Ich bin {name}, {alter} Jahre alt und komme aus {stadt}.")  
  
vorstellen("Anna", 25, "Berlin")  
# Ausgabe: Ich bin Anna, 25 Jahre alt und komme aus Berlin.  
  
# Falsche Reihenfolge führt zu falschem Ergebnis:  
vorstellen("Berlin", 25, "Anna")  
# Ausgabe: Ich bin Berlin, 25 Jahre alt und komme aus Anna.
```

6.3.2 Schlüsselwort-Parameter (Keyword Arguments)

Sie können Parameter explizit benennen:

```
def vorstellen(name, alter, stadt):  
    print(f"Ich bin {name}, {alter} Jahre alt und komme aus {stadt}.")  
  
# Mit Keyword Arguments (Reihenfolge egal):  
vorstellen(alter=30, name="Max", stadt="München")  
vorstellen(stadt="Hamburg", name="Lisa", alter=28)  
  
# Mischung aus Positions- und Keyword-Argumenten:  
vorstellen("Tom", stadt="Köln", alter=35)
```

6.3.3 Standardwerte (Default Parameters)

Parameter können Standardwerte haben:

```
def begruesse(name, gruß="Hallo"):  
    """Begrüßt eine Person mit einem optionalen Gruß."""  
    print(f"{gruß}, {name}!")  
  
begruesse("Anna")                # Hallo, Anna!  
begruesse("Max", "Guten Morgen") # Guten Morgen, Max!  
begruesse("Lisa", gruß="Hi")     # Hi, Lisa!
```

Praktisches Beispiel:


```
def potenz(basis, exponent=2):
    """Berechnet basis hoch exponent (Standard: Quadrat)."""
    return basis ** exponent

print(potenz(5))          # 25 (52)
print(potenz(5, 3))      # 125 (53)
print(potenz(2, 10))     # 1024 (210)
```

6.3.4 Mehrere Rückgabewerte

Funktionen können mehrere Werte als Tuple zurückgeben:

```
def min_max(zahlen):
    """Gibt das Minimum und Maximum einer Liste zurück."""
    return min(zahlen), max(zahlen)

# Rückgabewerte entpacken:
minimum, maximum = min_max([3, 7, 1, 9, 4])
print(f"Min: {minimum}, Max: {maximum}") # Min: 1, Max: 9

# Oder als Tuple verwenden:
ergebnis = min_max([10, 20, 5, 15])
print(ergebnis) # (5, 20)
```

Weiteres Beispiel:

```
def kreisberechnung(radius):
    """Berechnet Umfang und Fläche eines Kreises."""
    pi = 3.14159
    umfang = 2 * pi * radius
    flaeche = pi * radius ** 2
    return umfang, flaeche

u, f = kreisberechnung(5)
print(f"Umfang: {u:.2f}, Fläche: {f:.2f}")
# Ausgabe: Umfang: 31.42, Fläche: 78.54
```

6.3.5 Funktionen ohne Rückgabewert

Funktionen ohne return geben implizit None zurück:

```
def ausgabe(text):
    print(text)

ergebnis = ausgabe("Hallo")
print(ergebnis) # None
```

6.4 Lokale vs. globale Variablen

Variablen haben unterschiedliche Gültigkeitsbereiche (Scopes).

6.4.1 Lokale Variablen

Variablen, die innerhalb einer Funktion definiert werden, sind lokal:

```
def meine_funktion():
    x = 10  # Lokale Variable
    print(f"Innerhalb der Funktion: x = {x}")

meine_funktion()  # Innerhalb der Funktion: x = 10

# print(x)  # Fehler: NameError - x existiert außerhalb nicht
```

6.4.2 Globale Variablen

Variablen, die außerhalb von Funktionen definiert werden, sind global:

```
y = 20  # Globale Variable

def zeige_global():
    print(f"y in Funktion: {y}")

zeige_global()  # y in Funktion: 20
print(f"y außerhalb: {y}")  # y außerhalb: 20
```

6.4.3 Lokale Variable verdeckt globale

```
x = 100  # Globale Variable

def funktion():
    x = 50  # Lokale Variable (verdeckt globale)
    print(f"Lokal: {x}")

funktion()  # Lokal: 50
print(f"Global: {x}")  # Global: 100 (unverändert)
```

6.4.4 Globale Variablen ändern (nicht empfohlen)

```
counter = 0  # Globale Variable

def inkrementiere():
    global counter  # Explizit als global deklarieren
    counter += 1

print(counter)  # 0
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```

inkrementiere()
print(counter)    # 1
inkrementiere()
print(counter)    # 2

```

Warnung: Die Verwendung von `global` sollte vermieden werden, da sie zu schwer nachvollziehbarem Code führt. Besser: Rückgabewerte verwenden.

```

# Besser: Ohne global
def inkrementiere(wert):
    return wert + 1

counter = 0
counter = inkrementiere(counter)    # 1
counter = inkrementiere(counter)    # 2
print(counter)    # 2

```

6.5 Best Practices: DRY-Prinzip (Don't Repeat Yourself)

Das DRY-Prinzip besagt: Schreiben Sie keinen Code mehrfach. Nutzen Sie stattdessen Funktionen.

6.5.1 Schlechtes Beispiel (Code-Wiederholung)

```

# Preisberechnung für verschiedene Produkte:
preis1 = 100
rabatt1 = preis1 * 0.1
endpreis1 = preis1 - rabatt1
print(f"Endpreis: {endpreis1} Euro")

preis2 = 250
rabatt2 = preis2 * 0.1
endpreis2 = preis2 - rabatt2
print(f"Endpreis: {endpreis2} Euro")

preis3 = 75
rabatt3 = preis3 * 0.1
endpreis3 = preis3 - rabatt3
print(f"Endpreis: {endpreis3} Euro")

```

6.5.2 Gutes Beispiel (mit Funktion)

```

def berechne_endpreis(preis, rabatt_prozent=10):
    """Berechnet den Endpreis nach Rabattabzug."""
    rabatt = preis * (rabatt_prozent / 100)
    endpreis = preis - rabatt
    return endpreis

print(f"Endpreis: {berechne_endpreis(100)} Euro")

```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
print(f"Endpreis: {berechne_endpreis(250)} Euro")
print(f"Endpreis: {berechne_endpreis(75)} Euro")

# Oder mit unterschiedlichen Rabatten:
print(f"Endpreis: {berechne_endpreis(200, 20)} Euro") # 20% Rabatt
```

6.5.3 Funktionen für komplexe Logik

```
def ist_schaltjahr(jahr):
    """Prüft, ob ein Jahr ein Schaltjahr ist."""
    if jahr % 4 != 0:
        return False
    elif jahr % 100 != 0:
        return True
    elif jahr % 400 != 0:
        return False
    else:
        return True

# Verwendung:
print(ist_schaltjahr(2024)) # True
print(ist_schaltjahr(2100)) # False
print(ist_schaltjahr(2000)) # True

# Statt die Logik mehrfach zu schreiben:
jahre = [2020, 2021, 2022, 2023, 2024]
for jahr in jahre:
    if ist_schaltjahr(jahr):
        print(f"{jahr} ist ein Schaltjahr")
```

6.5.4 Funktionen zur Code-Organisation

```
def eingabe_validieren(wert, minimum, maximum):
    """Prüft, ob ein Wert im gültigen Bereich liegt."""
    return minimum <= wert <= maximum

def temperatur_einlesen():
    """Liest eine gültige Temperatur ein."""
    while True:
        temp = float(input("Temperatur (-50 bis 50°C): "))
        if eingabe_validieren(temp, -50, 50):
            return temp
        else:
            print("Ungültiger Wert!")

def temperatur_bewerten(temp):
    """Bewertet eine Temperatur."""
    if temp < 0:
        return "Kalt"
    elif temp < 20:
        return "Kühl"
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
elif temp < 30:
    return "Warm"
else:
    return "Heiß"

# Hauptprogramm:
temperatur = temperatur_einlesen()
bewertung = temperatur_bewerten(temperatur)
print(f"Es ist {bewertung}.")
```

6.6 Praktische Beispiele

6.6.1 Beispiel 1: BMI-Rechner mit Funktionen

```
def bmi_berechnen(gewicht, groesse):
    """Berechnet den Body Mass Index.

    Args:
        gewicht: Gewicht in kg
        groesse: Größe in Metern

    Returns:
        BMI als Float
    """
    return gewicht / (groesse ** 2)

def bmi_kategorie(bmi):
    """Gibt die BMI-Kategorie zurück."""
    if bmi < 18.5:
        return "Untergewicht"
    elif bmi < 25:
        return "Normalgewicht"
    elif bmi < 30:
        return "Übergewicht"
    else:
        return "Adipositas"

def bmi_programm():
    """Führt BMI-Berechnung durch."""
    print("=== BMI-Rechner ===")
    gewicht = float(input("Gewicht in kg: "))
    groesse = float(input("Größe in m: "))

    bmi = bmi_berechnen(gewicht, groesse)
    kategorie = bmi_kategorie(bmi)

    print(f"\nIhr BMI: {bmi:.1f}")
    print(f"Kategorie: {kategorie}")

# Programm ausführen:
bmi_programm()
```

6.6.2 Beispiel 2: Notenverwaltung

```
def durchschnitt_berechnen(noten):
    """Berechnet den Notendurchschnitt."""
    if len(noten) == 0:
        return 0
    return sum(noten) / len(noten)

def note_ist_bestanden(note):
    """Prüft, ob eine Note bestanden ist (Note <= 4)."""
    return note <= 4

def notenstatistik(noten):
    """Erstellt eine Statistik über Noten."""
    durchschnitt = durchschnitt_berechnen(noten)
    bestanden = sum(1 for note in noten if note_ist_bestanden(note))
    nicht_bestanden = len(noten) - bestanden

    return {
        "durchschnitt": durchschnitt,
        "bestanden": bestanden,
        "nicht_bestanden": nicht_bestanden,
        "anzahl": len(noten)
    }

# Verwendung:
noten = [1, 2, 3, 2, 4, 5, 3, 1]
statistik = notenstatistik(noten)

print(f"Anzahl Noten: {statistik['anzahl']}")
print(f"Durchschnitt: {statistik['durchschnitt']:.2f}")
print(f"Bestanden: {statistik['bestanden']}")
print(f"Nicht bestanden: {statistik['nicht_bestanden']}")
```

6.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- Funktionen mit `def` definieren und aufrufen
- Parameter übergeben (Positions- und Keyword-Parameter)
- Standardwerte für Parameter festlegen
- Werte mit `return` zurückgeben
- Den Unterschied zwischen lokalen und globalen Variablen
- Das DRY-Prinzip (Don't Repeat Yourself) anwenden

Im nächsten Kapitel lernen Sie die Python Standard Library kennen, die viele nützliche Module bereitstellt.

Python Standard Library

Die Python Standard Library ist eine umfangreiche Sammlung von Modulen, die mit Python mitgeliefert werden. Sie müssen nicht separat installiert werden und bieten Funktionen für viele gängige Aufgaben. Diese Module decken Bereiche wie Dateisystemoperationen, Zeitberechnungen, mathematische Funktionen, Zufallszahlen und vieles mehr ab.

Die Verwendung der Standard Library hat mehrere Vorteile: Sie ist sofort verfügbar, gut dokumentiert, plattformübergreifend einsetzbar und von der Python-Community getestet. Anstatt eigene Lösungen für häufige Probleme zu entwickeln, können Sie auf bewährte Module zurückgreifen.

7.1 Wichtige Module: `os`, `sys`, `pathlib`, `datetime`

Python kommt mit einer Vielzahl nützlicher Module, die den Alltag eines Entwicklers erheblich erleichtern. In diesem Abschnitt lernen Sie die wichtigsten Module kennen, die für die Arbeit mit dem Betriebssystem, Dateipfaden und Zeitberechnungen unerlässlich sind.

7.1.1 Das `os`-Modul

Das `os`-Modul bietet Funktionen für die Interaktion mit dem Betriebssystem. Es ermöglicht Ihnen, plattformunabhängig auf Dateisysteme zuzugreifen, Verzeichnisse zu navigieren und Umgebungsvariablen auszulesen. Das Modul abstrahiert viele betriebssystemspezifische Details, sodass Ihr Code auf Windows, Linux und macOS funktioniert.

Ein häufiger Anwendungsfall ist das Arbeiten mit Dateipfaden. Mit `os.path.join()` können Sie Pfade plattformunabhängig zusammensetzen - auf Windows werden Backslashes (`\`), auf Unix-Systemen Slashes (`/`) verwendet. Die Funktion `os.getcwd()` gibt das aktuelle Arbeitsverzeichnis zurück, was besonders nützlich ist, wenn Sie relative Pfade verwenden.

```
import os

# Aktuelles Arbeitsverzeichnis anzeigen:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
aktuelles_verzeichnis = os.getcwd()
print(f"Aktuelles Verzeichnis: {aktuelles_verzeichnis}")

# Dateien und Ordner im Verzeichnis auflisten:
inhalt = os.listdir(".")
print(f"Inhalt: {inhalt}")

# Prüfen, ob Datei oder Ordner existiert:
if os.path.exists("datei.txt"):
    print("Datei existiert")
else:
    print("Datei existiert nicht")

# Prüfen, ob es sich um eine Datei handelt:
if os.path.isfile("dokument.pdf"):
    print("Es ist eine Datei")

# Prüfen, ob es sich um einen Ordner handelt:
if os.path.isdir("ordner"):
    print("Es ist ein Ordner")

# Dateipfade zusammenfügen:
pfad = os.path.join("ordner", "unterordner", "datei.txt")
print(pfad) # ordner/unterordner/datei.txt (oder ordner\unterordner\datei.
            txt auf Windows)
```

Die Funktionen `os.path.exists()`, `os.path.isfile()` und `os.path.isdir()` sind unverzichtbar, wenn Sie prüfen möchten, ob eine Datei oder ein Verzeichnis existiert, bevor Sie darauf zugreifen. Dies verhindert Fehler und macht Ihren Code robuster. Die Funktion `os.listdir()` gibt eine Liste aller Dateien und Ordner in einem Verzeichnis zurück, was für das Durchsuchen von Verzeichnissen nützlich ist.

Umgebungsvariablen:

Umgebungsvariablen sind systemweite Einstellungen, die Informationen wie Benutzernamen, Home-Verzeichnisse oder Konfigurationspfade speichern. Mit dem `os`-Modul können Sie diese Variablen auslesen, um systemspezifische Informationen zu erhalten oder Konfigurationen zu laden.

```
import os

# Umgebungsvariable lesen:
home = os.environ.get("HOME")
print(f"Home-Verzeichnis: {home}")

# Alle Umgebungsvariablen:
for key, value in os.environ.items():
    print(f"{key}: {value}")
```

Die Methode `os.environ.get()` ist sicherer als direkter Zugriff, da sie `None` zurückgibt, wenn die Variable nicht existiert, anstatt einen Fehler zu werfen. Sie können auch einen Standardwert als zweiten Parameter angeben: `os.environ.get("MEINE_VAR", "Standard")`.

7.1.2 Das sys-Modul

Das `sys`-Modul bietet Zugriff auf systembezogene Funktionen und Parameter, die eng mit dem Python-Interpreter verbunden sind. Es ist besonders nützlich, wenn Sie Informationen über die Python-Laufzeitumgebung benötigen oder Ihr Programm gezielt beenden möchten.

Mit `sys.argv` können Sie Kommandozeilenargumente auslesen, die beim Aufruf eines Python-Skripts übergeben werden. Dies ist die Grundlage für einfache Kommandozeilenprogramme. Das erste Element `sys.argv[0]` enthält immer den Namen des Skripts selbst. Mit `sys.exit()` können Sie Ihr Programm mit einem bestimmten Exit-Code beenden - 0 bedeutet Erfolg, andere Werte signalisieren Fehler.

```
import sys

# Python-Version:
print(f"Python-Version: {sys.version}")

# Plattform:
print(f"Plattform: {sys.platform}") # z.B. 'linux', 'win32', 'darwin'

# Kommandozeilenargumente:
print(f"Script-Name: {sys.argv[0]}")
print(f"Alle Argumente: {sys.argv}")

# Python-Suchpfade für Module:
print("Module-Pfade:")
for pfad in sys.path:
    print(f"    {pfad}")
```

Die Liste `sys.path` zeigt alle Verzeichnisse, in denen Python nach Modulen sucht, wenn Sie `import` verwenden. Dies ist hilfreich beim Debuggen von Import-Problemen. Die Plattform-Information (`sys.platform`) hilft Ihnen, plattformspezifischen Code zu schreiben, falls nötig.

7.1.3 Das pathlib-Modul (Modern)

`pathlib` ist die moderne, objektorientierte Alternative zu `os.path` und wurde mit Python 3.4 eingeführt. Es bietet eine elegantere und intuitivere Syntax für die Arbeit mit Dateipfaden. Anstatt Funktionen wie `os.path.join()` zu verwenden, arbeiten Sie mit `Path`-Objekten, die mit dem `/`-Operator verkettet werden können.

Die Objektorientierung macht den Code lesbarer: `pfad.exists()` ist selbsterklärender als `os.path.exists(pfad)`. Zudem bietet `pathlib` viele nützliche Methoden wie `.glob()` für das Suchen von Dateien mit Wildcards oder `.read_text()` für das direkte Lesen von Dateiinhalten. Für neue Projekte sollten Sie `pathlib` gegenüber `os.path` bevorzugen.

```
from pathlib import Path

# Aktuelles Verzeichnis:
aktuell = Path.cwd()
print(f"Aktuelles Verzeichnis: {aktuell}")

# Home-Verzeichnis:
home = Path.home()
print(f"Home-Verzeichnis: {home}")
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# Pfade zusammenfügen:
pfad = Path("ordner") / "unterordner" / "datei.txt"
print(pfad)

# Prüfungen:
datei = Path("beispiel.txt")
print(f"Existiert: {datei.exists()}")
print(f"Ist Datei: {datei.is_file()}")
print(f"Ist Ordner: {datei.is_dir()}")

# Dateiinformationen:
if datei.exists():
    print(f"Dateiname: {datei.name}")
    print(f"Erweiterung: {datei.suffix}")
    print(f"Ohne Erweiterung: {datei.stem}")
    print(f"Elternverzeichnis: {datei.parent}")

# Alle Python-Dateien in einem Verzeichnis finden:
verzeichnis = Path(".")
for datei in verzeichnis.glob("*.py"):
    print(datei)
```

Die `.glob()`-Methode ist besonders praktisch für das Filtern von Dateien. Sie können Wildcards wie `*.py` (alle Python-Dateien) oder `**/*.txt` (alle Textdateien rekursiv) verwenden. Die Eigenschaften `.name`, `.stem`, `.suffix` und `.parent` machen es einfach, Dateinamen zu analysieren und zu manipulieren, ohne String-Operationen durchführen zu müssen.

7.1.4 Das datetime-Modul

Das `datetime`-Modul ermöglicht die Arbeit mit Datum und Zeit. Es bietet Klassen für die Darstellung von Zeitpunkten, Datumsangaben und Zeitdifferenzen. Die Arbeit mit Datum und Zeit ist in vielen Anwendungen wichtig: von der Protokollierung von Ereignissen über die Berechnung von Fristen bis hin zur Planung von Aufgaben.

Python unterscheidet zwischen `date` (nur Datum), `time` (nur Uhrzeit) und `datetime` (Datum und Uhrzeit kombiniert). Die Klasse `timedelta` repräsentiert Zeitdifferenzen und ermöglicht Berechnungen wie "3 Tage später" oder "2 Stunden früher". Mit der `strftime()`-Methode können Sie Datumsangaben in verschiedenen Formaten ausgeben - zum Beispiel "14.01.2026" oder "Mittwoch, 14. Januar 2026".

```
from datetime import datetime, date, time, timedelta

# Aktuelles Datum und Zeit:
jetzt = datetime.now()
print(f"Jetzt: {jetzt}")
print(f"Jahr: {jetzt.year}")
print(f"Monat: {jetzt.month}")
print(f"Tag: {jetzt.day}")
print(f"Stunde: {jetzt.hour}")
print(f"Minute: {jetzt.minute}")

# Nur Datum:
heute = date.today()
print(f"Heute: {heute}")
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```

# Datum erstellen:
geburtstag = date(1990, 5, 15)
print(f"Geburtstag: {geburtstag}")

# Formatierte Ausgabe:
print(jetzt.strftime("%d.%m.%Y %H:%M:%S")) # 14.01.2026 15:30:45
print(heute.strftime("%A, %d. %B %Y"))      # Mittwoch, 14. Januar 2026

# Zeitdifferenzen:
morgen = heute + timedelta(days=1)
print(f"Morgen: {morgen}")

in_einer_woche = heute + timedelta(weeks=1)
print(f"In einer Woche: {in_einer_woche}")

# Differenz zwischen zwei Daten:
differenz = heute - geburtstag
print(f"Tage seit Geburtstag: {differenz.days}")

```

Die Formatierungs-codes für `strftime()` sind standardisiert: `%Y` für vierstelliges Jahr, `%m` für Monat, `%d` für Tag, `%H` für Stunde, `%M` für Minute, `%S` für Sekunde. Mit `%A` erhalten Sie den ausgeschriebenen Wochentag, mit `%B` den Monatsnamen. Zeitdifferenzen können Sie einfach mit `+` und `-` berechnen, was den Code sehr lesbar macht.

Praktisches Beispiel - Alter berechnen:

Das folgende Beispiel zeigt, wie Sie das Alter einer Person in Jahren berechnen. Die Herausforderung besteht darin, zu berücksichtigen, ob der Geburtstag im aktuellen Jahr bereits stattgefunden hat. Wenn nicht, müssen wir das Alter um eins reduzieren. Diese Art von Logik ist typisch für reale Anwendungen der `datetime`-Berechnungen.

```

from datetime import date

def berechne_alter(geburtsdatum):
    """Berechnet das Alter in Jahren."""
    heute = date.today()
    alter = heute.year - geburtsdatum.year

    # Korrektur, falls Geburtstag noch nicht war:
    if (heute.month, heute.day) < (geburtsdatum.month, geburtsdatum.day):
        alter -= 1

    return alter

geburtstag = date(1995, 8, 20)
alter = berechne_alter(geburtstag)
print(f"Alter: {alter} Jahre")

```

7.2 Module importieren und verwenden

Python bietet verschiedene Möglichkeiten, Module zu importieren. Die Wahl der richtigen Import-Methode beeinflusst die Lesbarkeit und Wartbarkeit Ihres Codes. Es ist wichtig zu verstehen, wann welche Import-Variante sinnvoll ist.

7.2.1 Import-Varianten

Es gibt vier Hauptmethoden, um Module zu importieren. Jede hat ihre Vor- und Nachteile, abhängig vom Anwendungsfall. Die Wahl der Import-Methode sollte immer dem Ziel dienen, den Code klar und verständlich zu halten.

Gesamtes Modul importieren:

Die Standard-Import-Methode importiert das gesamte Modul. Sie müssen dann den Modulnamen als Präfix verwenden, wenn Sie Funktionen aufrufen. Dies hat den Vorteil, dass immer klar ist, woher eine Funktion stammt, und verhindert Namenskonflikte.

```
import math

print(math.pi)           # 3.141592653589793
print(math.sqrt(16))     # 4.0
print(math.cos(0))       # 1.0
```

Modul mit Alias importieren:

Wenn ein Modulname sehr lang ist oder oft verwendet wird, können Sie einen kürzeren Alias definieren. Dies ist besonders bei Modulen wie `numpy` (oft als `np` importiert) oder `pandas` (oft als `pd`) üblich. Der Alias sollte jedoch immer sinnvoll und in der Community etabliert sein, um die Lesbarkeit zu gewährleisten.

```
import math as m

print(m.pi)              # 3.141592653589793
print(m.sqrt(25))        # 5.0
```

Bestimmte Funktionen importieren:

Wenn Sie nur wenige spezifische Funktionen aus einem Modul benötigen, können Sie diese direkt importieren. Der Vorteil ist kürzerer Code, da Sie den Modulnamen nicht als Präfix schreiben müssen. Verwenden Sie diese Methode jedoch nur, wenn klar ist, woher die Funktionen stammen, und wenn keine Namenskonflikte entstehen können.

```
from math import pi, sqrt, cos

print(pi)                # 3.141592653589793
print(sqrt(9))           # 3.0
print(cos(0))            # 1.0
```

Alles aus einem Modul importieren (nicht empfohlen):

Die `import *`-Syntax importiert alle Funktionen und Variablen aus einem Modul. Dies wird jedoch nicht empfohlen, da es zu Namenskonflikten führen kann und nicht klar ist, welche Namen importiert wurden. Es erschwert die Wartung und das Debugging. Verwenden Sie diese Methode nur in Ausnahmefällen, etwa beim interaktiven Experimentieren in der Python-Shell.

```
from math import *

print(pi)           # 3.141592653589793
print(sqrt(4))      # 2.0
# Problem: Namenskonflikte möglich!
```

7.2.2 Wichtige Module im Überblick

Die Standard Library enthält zahlreiche spezialisierte Module für verschiedene Aufgaben. Im Folgenden werden die am häufigsten verwendeten Module vorgestellt, die in fast jedem Python-Projekt nützlich sind.

math - Mathematische Funktionen:

Das `math`-Modul bietet mathematische Funktionen und Konstanten, die über die eingebauten Operatoren hinausgehen. Es enthält trigonometrische Funktionen, Logarithmen, Wurzeln und Konstanten wie `Pi` und Euler's Zahl. Dieses Modul ist unverzichtbar für wissenschaftliche Berechnungen, Geometrie und statistische Analysen.

```
import math

print(math.sqrt(16))      # Quadratwurzel: 4.0
print(math.pow(2, 8))     # Potenz: 256.0
print(math.floor(3.7))    # Abrunden: 3
print(math.ceil(3.2))     # Aufrunden: 4
print(math.sin(math.pi))  # Sinus: 1.2246467991473532e-16 (~ 0)
print(math.log(10))       # Natürlicher Logarithmus: 2.302585092994046
```

Die Funktionen `floor()` und `ceil()` sind nützlich für das Auf- und Abrunden, während `pow()` eine Alternative zum `**`-Operator darstellt. Beachten Sie, dass `math` nur für reelle Zahlen funktioniert - für komplexe Zahlen verwenden Sie das `cmath`-Modul.

random - Zufallszahlen:

Das `random`-Modul erzeugt Pseudozufallszahlen für verschiedene Verteilungen. Es ist ideal für Simulationen, Spiele, zufällige Auswahlen oder das Mischen von Daten. Beachten Sie, dass die erzeugten Zahlen nicht für kryptographische Zwecke geeignet sind - verwenden Sie dafür das `secrets`-Modul.

```
import random

# Zufällige Ganzzahl:
print(random.randint(1, 10))      # z.B. 7

# Zufällige Fließkommazahl:
print(random.random())            # z.B. 0.734... (0.0 bis 1.0)

# Zufällige Auswahl aus Liste:
farben = ["rot", "blau", "grün"]
print(random.choice(farben))      # z.B. "blau"

# Liste mischen:
zahlen = [1, 2, 3, 4, 5]
random.shuffle(zahlen)
print(zahlen)                     # z.B. [3, 1, 5, 2, 4]
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# Mehrere zufällige Elemente:
print(random.sample([1, 2, 3, 4, 5], 3)) # z.B. [2, 5, 1]
```

Die Funktionen `choice()` und `sample()` unterscheiden sich darin, dass `choice()` ein einzelnes Element zurückgibt, während `sample()` mehrere Elemente ohne Zurücklegen auswählt. Die Funktion `shuffle()` verändert die ursprüngliche Liste direkt (in-place), während `sample()` eine neue Liste zurückgibt.

json - JSON-Daten verarbeiten:

JSON (JavaScript Object Notation) ist ein weit verbreitetes Datenaustauschformat, das in Webdiensten, APIs und Konfigurationsdateien verwendet wird. Das `json`-Modul ermöglicht die Konvertierung zwischen Python-Datenstrukturen (Dictionaries, Listen) und JSON-Strings. Dies ist essentiell für die Kommunikation mit Web-APIs oder das Speichern strukturierter Daten.

```
import json

# Python-Dictionary zu JSON:
daten = {
    "name": "Anna",
    "alter": 25,
    "hobbys": ["Lesen", "Sport"]
}

json_string = json.dumps(daten, indent=2)
print(json_string)
# {
#   "name": "Anna",
#   "alter": 25,
#   "hobbys": ["Lesen", "Sport"]
# }

# JSON zu Python-Dictionary:
json_text = '{"name": "Max", "alter": 30}'
person = json.loads(json_text)
print(person["name"]) # Max
```

Die Funktion `json.dumps()` konvertiert Python-Objekte in JSON-Strings (Serialisierung), während `json.loads()` JSON-Strings in Python-Objekte umwandelt (Deserialisierung). Der Parameter `indent` bei `dumps()` macht die Ausgabe für Menschen lesbar. Für das direkte Lesen und Schreiben von JSON-Dateien verwenden Sie `json.dump()` und `json.load()` (ohne das "s").

collections - Erweiterte Datenstrukturen:

Das `collections`-Modul bietet spezialisierte Container-Datentypen, die über die eingebauten `list`, `dict`, `set` und `tuple` hinausgehen. Diese Datenstrukturen lösen häufige Programmierprobleme effizient und elegant. Der `Counter` ist besonders nützlich für Häufigkeitsanalysen, während `defaultdict` das Arbeiten mit verschachtelten Datenstrukturen vereinfacht.

```
from collections import Counter, defaultdict

# Counter - Elemente zählen:
buchstaben = "abracadabra"
zaehler = Counter(buchstaben)
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
print(zaehler)  # Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
print(zaehler.most_common(2))  # [('a', 5), ('b', 2)]

# defaultdict - Dictionary mit Standardwert:
noten = defaultdict(list)
noten["Anna"].append(1)
noten["Anna"].append(2)
noten["Max"].append(3)
print(dict(noten))  # {'Anna': [1, 2], 'Max': [3]}
```

Der Counter zählt automatisch, wie oft jedes Element in einer Sequenz vorkommt. Die Methode `most_common(n)` gibt die `n` häufigsten Elemente zurück. Ein `defaultdict` vermeidet `KeyError`-Exceptions, indem es automatisch einen Standardwert für neue Schlüssel erstellt - im Beispiel eine leere Liste. Dies ist besonders praktisch beim Gruppieren von Daten.

7.3 Praktisches Beispiel

7.3.1 Datei-Backup mit Zeitstempel

Das folgende praktische Beispiel kombiniert mehrere Module der Standard Library: `shutil` für Dateioperationen, `datetime` für Zeitstempel und `pathlib` für die Pfadverwaltung. Es zeigt, wie Sie ein automatisches Backup-System erstellen können, das Dateien mit einem Zeitstempel versieht. Dies ist ein typisches Beispiel für die Kombination verschiedener Module, um eine nützliche Funktionalität zu implementieren.

```
import shutil
from datetime import datetime
from pathlib import Path

def backup_erstellen(quelldatei):
    """Erstellt ein Backup einer Datei mit Zeitstempel."""
    quelle = Path(quelldatei)

    if not quelle.exists():
        print(f"Fehler: {quelldatei} existiert nicht!")
        return

    # Zeitstempel erstellen:
    zeitstempel = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Backup-Dateinamen erstellen:
    backup_name = f"{quelle.stem}_backup_{zeitstempel}{quelle.suffix}"
    backup_pfad = quelle.parent / backup_name

    # Datei kopieren:
    shutil.copy2(quelle, backup_pfad)
    print(f"Backup erstellt: {backup_pfad}")

# Verwendung:
# backup_erstellen("wichtige_datei.txt")
# Erstellt z.B.: wichtige_datei_backup_20260114_153045.txt
```

Die Funktion prüft zunächst, ob die Quelldatei existiert, um Fehler zu vermeiden. Der Zeitstempel wird im Format `YYYYMMDD_HHMMSS` erstellt, was eine chronologische Sortierung ermöglicht. Mit `pathlib` wird der neue Dateiname elegant zusammengesetzt: `stem` gibt den Dateinamen ohne Erweiterung zurück, `suffix` die Erweiterung. Die Funktion `shutil.copy2()` kopiert die Datei inklusive Metadaten (Zeitstempel, Berechtigungen).

7.4 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- Wichtige Module der Standard Library: `os`, `sys`, `pathlib`, `datetime`
- Wie Module importiert werden: `import`, `from ... import`, Aliase
- Praktische Module: `math`, `random`, `json`, `collections`
- Anwendungsbeispiele für verschiedene Aufgaben

Im nächsten Kapitel lernen Sie, wie Sie mit Dateien arbeiten: lesen, schreiben und verarbeiten.

Dateioperationen

In diesem Kapitel lernen Sie, wie Sie Dateien lesen, schreiben und verarbeiten. Dateioperationen sind essenziell für viele Programme.

8.1 Dateien lesen und schreiben

Python bietet einfache Möglichkeiten, mit Textdateien zu arbeiten.

8.1.1 Datei zum Lesen öffnen

Die `open()`-Funktion öffnet eine Datei:

```
# Datei öffnen und lesen:
datei = open("beispiel.txt", "r") # "r" = read (lesen)
inhalt = datei.read()
print(inhalt)
datei.close() # Wichtig: Datei schließen!
```

Verschiedene Lesemethoden:

```
# Gesamten Inhalt lesen:
datei = open("beispiel.txt", "r")
alles = datei.read()
datei.close()

# Datei zeilenweise lesen:
datei = open("beispiel.txt", "r")
for zeile in datei:
    print(zeile, end="") # end="" verhindert doppelten Zeilenumbruch
datei.close()

# Alle Zeilen als Liste lesen:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
datei = open("beispiel.txt", "r")
zeilen = datei.readlines()  # Liste von Strings
datei.close()

for zeile in zeilen:
    print(zeile.strip())  # strip() entfernt Zeilenumbrüche
```

8.1.2 Datei zum Schreiben öffnen

```
# Neue Datei erstellen oder überschreiben:
datei = open("ausgabe.txt", "w")  # "w" = write (schreiben)
datei.write("Hallo Welt!\n")
datei.write("Dies ist eine neue Zeile.\n")
datei.close()

# An bestehende Datei anhängen:
datei = open("ausgabe.txt", "a")  # "a" = append (anhängen)
datei.write("Diese Zeile wird angehängt.\n")
datei.close()
```

Modi im Überblick:

- "r" - Lesen (Datei muss existieren)
- "w" - Schreiben (überschreibt existierende Datei)
- "a" - Anhängen (fügt am Ende hinzu)
- "r+" - Lesen und Schreiben
- "rb" - Binärdatei lesen
- "wb" - Binärdatei schreiben

8.1.3 Best Practice: Dateien mit Context Manager öffnen

Mit `with` müssen Sie die Datei nicht manuell schließen:

```
# Lesen mit Context Manager:
with open("beispiel.txt", "r") as datei:
    inhalt = datei.read()
    print(inhalt)
# Datei wird automatisch geschlossen!

# Schreiben mit Context Manager:
with open("ausgabe.txt", "w") as datei:
    datei.write("Zeile 1\n")
    datei.write("Zeile 2\n")
    datei.write("Zeile 3\n")
```

Vorteile des Context Managers:

- Datei wird automatisch geschlossen

- Auch bei Fehlern wird die Datei korrekt geschlossen
- Sauberer und lesbarer Code

8.1.4 Praktisches Beispiel: Datei zeilenweise verarbeiten

```
# Zahlen aus Datei lesen und summieren:
summe = 0

with open("zahlen.txt", "r") as datei:
    for zeile in datei:
        zahl = int(zeile.strip())
        summe += zahl

print(f"Summe: {summe}")
```

8.2 Context Manager (with-Statement)

Der Context Manager stellt sicher, dass Ressourcen (wie Dateien) ordnungsgemäß geschlossen werden.

8.2.1 Ohne Context Manager (alt, fehleranfällig)

```
# Riskant - was passiert bei einem Fehler?
datei = open("datei.txt", "r")
try:
    inhalt = datei.read()
    # Verarbeitung...
finally:
    datei.close() # Sicherstellen, dass Datei geschlossen wird
```

8.2.2 Mit Context Manager (modern, sicher)

```
# Sicher und elegant:
with open("datei.txt", "r") as datei:
    inhalt = datei.read()
    # Verarbeitung...
# Datei wird automatisch geschlossen
```

8.2.3 Mehrere Dateien gleichzeitig

```
# Mehrere Dateien gleichzeitig öffnen:
with open("eingabe.txt", "r") as quelle, open("ausgabe.txt", "w") as ziel:
    for zeile in quelle:
        # Zeile verarbeiten und in Zielfeile schreiben:
        verarbeitet = zeile.upper() # Großbuchstaben
        ziel.write(verarbeitet)
```

8.2.4 Eigene Context Manager erstellen (Vorschau)

```
class DateiLogger:
    """Einfacher Context Manager für Logging."""

    def __init__(self, dateiname):
        self.dateiname = dateiname
        self.datei = None

    def __enter__(self):
        self.datei = open(self.dateiname, "a")
        return self.datei

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.datei:
            self.datei.close()

# Verwendung:
with DateiLogger("log.txt") as log:
    log.write("Programm gestartet\n")
    log.write("Vorgang abgeschlossen\n")
```

8.3 CSV-Dateien verarbeiten

CSV (Comma-Separated Values) ist ein häufiges Format für tabellarische Daten.

8.3.1 CSV manuell lesen

```
# Einfaches CSV-Lesen:
with open("daten.csv", "r") as datei:
    for zeile in datei:
        spalten = zeile.strip().split(",")
        print(spalten)

# Beispiel-CSV-Inhalt:
# Name,Alter,Stadt
# Anna,25,Berlin
# Max,30,München

# Ausgabe:
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
# ['Name', 'Alter', 'Stadt']  
# ['Anna', '25', 'Berlin']  
# ['Max', '30', 'München']
```

8.3.2 CSV-Modul verwenden (empfohlen)

Das `csv`-Modul bietet robustere Funktionen:

```
import csv  
  
# CSV-Datei lesen:  
with open("daten.csv", "r") as datei:  
    reader = csv.reader(datei)  
  
    # Header überspringen:  
    header = next(reader)  
    print(f"Spalten: {header}")  
  
    # Daten lesen:  
    for zeile in reader:  
        name, alter, stadt = zeile  
        print(f"{name} ist {alter} Jahre alt und wohnt in {stadt}")
```

8.3.3 CSV mit DictReader (mit Spaltenüberschriften)

```
import csv  
  
# CSV als Dictionary lesen:  
with open("daten.csv", "r") as datei:  
    reader = csv.DictReader(datei)  
  
    for zeile in reader:  
        print(f"Name: {zeile['Name']}")  
        print(f"Alter: {zeile['Alter']}")  
        print(f"Stadt: {zeile['Stadt']}")  
        print()
```

8.3.4 CSV schreiben

```
import csv  
  
# CSV-Datei schreiben:  
personen = [  
    ["Name", "Alter", "Stadt"],  
    ["Anna", 25, "Berlin"],  
    ["Max", 30, "München"],  
    ["Lisa", 28, "Hamburg"]  
]
```

(Fortsetzung auf der nächsten Seite)

```
with open("ausgabe.csv", "w", newline="") as datei:
    writer = csv.writer(datei)

    for zeile in personen:
        writer.writerow(zeile)

# Mit DictWriter:
personen_dict = [
    {"Name": "Tom", "Alter": 35, "Stadt": "Köln"},
    {"Name": "Eva", "Alter": 27, "Stadt": "Dresden"}
]

with open("ausgabe2.csv", "w", newline="") as datei:
    feldnamen = ["Name", "Alter", "Stadt"]
    writer = csv.DictWriter(datei, fieldnames=feldnamen)

    writer.writeheader() # Header schreiben
    writer.writerows(personen_dict) # Alle Zeilen schreiben
```

8.3.5 Praktisches Beispiel: CSV-Statistik

```
import csv

def csv_statistik(dateiname):
    """Erstellt eine Statistik aus einer CSV-Datei mit Zahlen."""
    zahlen = []

    with open(dateiname, "r") as datei:
        reader = csv.DictReader(datei)

        for zeile in reader:
            # Annahme: Spalte "Wert" enthält Zahlen
            wert = float(zeile["Wert"])
            zahlen.append(wert)

    if zahlen:
        print(f"Anzahl: {len(zahlen)}")
        print(f"Summe: {sum(zahlen)}")
        print(f"Durchschnitt: {sum(zahlen) / len(zahlen):.2f}")
        print(f"Minimum: {min(zahlen)}")
        print(f"Maximum: {max(zahlen)}")

# Verwendung:
# csv_statistik("verkaufszahlen.csv")
```

8.4 Praktisches Beispiel: Einfaches Logbuch-System

Ein kompaktes Beispiel für ein Logbuch mit verschiedenen Log-Levels:

```
from datetime import datetime

class Logbuch:
    """Einfaches Logbuch-System."""

    def __init__(self, dateiname="logbuch.txt"):
        self.dateiname = dateiname

    def eintrag(self, nachricht, level="INFO"):
        """Fügt einen Logbuch-Eintrag hinzu."""
        zeitstempel = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        log_zeile = f"[{zeitstempel}] {level}: {nachricht}\n"

        with open(self.dateiname, "a") as datei:
            datei.write(log_zeile)

    def info(self, nachricht):
        """Info-Level Eintrag."""
        self.eintrag(nachricht, "INFO")

    def warnung(self, nachricht):
        """Warnungs-Level Eintrag."""
        self.eintrag(nachricht, "WARNUNG")

    def fehler(self, nachricht):
        """Fehler-Level Eintrag."""
        self.eintrag(nachricht, "FEHLER")

    def anzeigen(self, anzahl=10):
        """Zeigt die letzten n Einträge an."""
        try:
            with open(self.dateiname, "r") as datei:
                zeilen = datei.readlines()
                letzte_zeilen = zeilen[-anzahl:]

                print(f"=== Letzte {len(letzte_zeilen)} Einträge ===")
                for zeile in letzte_zeilen:
                    print(zeile, end="")
        except FileNotFoundError:
            print("Keine Logbuch-Datei gefunden.")

# Verwendung:
log = Logbuch()
log.info("Programm gestartet")
log.info("Datei verarbeitet")
log.warnung("Speicher wird knapp")
log.fehler("Datei nicht gefunden")
log.anzeigen()
```

8.5 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- Dateien mit `open()` lesen und schreiben
- Den Context Manager (`with`-Statement) für sicheres Dateihandling
- Modi: `"r"` (lesen), `"w"` (schreiben), `"a"` (anhängen)
- CSV-Dateien mit dem `csv`-Modul verarbeiten
- Praktische Anwendungen: Logbuch, Kontaktliste, Datei-Verschlüsselung

8.6 Ausblick

Sie haben nun die Grundlagen von Python kennengelernt:

- Variablen, Datentypen und Operatoren
- Verzweigungen und Schleifen
- Datenstrukturen (Listen, Dictionaries, Sets, Tuples)
- Funktionen und Code-Organisation
- Standard Library und Module
- Dateioperationen

Mit diesem Wissen können Sie bereits viele praktische Programme schreiben. Weiterführende Themen könnten sein:

- Objektorientierte Programmierung (Klassen und Objekte)
- Fehlerbehandlung und Exceptions
- Reguläre Ausdrücke
- Web-Entwicklung mit Flask oder Django
- Datenanalyse mit Pandas
- GUI-Programmierung mit Tkinter oder PyQt

Viel Erfolg beim Programmieren mit Python!

9.1 A. Grundlegende Kommandozeilen-Befehle

Diese Befehle funktionieren plattformübergreifend in Windows CMD, PowerShell und Unix-Shell (bash/zsh):

9.1.1 Aktuelles Verzeichnis anzeigen

```
# Windows CMD:  
cd  
  
# PowerShell, bash, zsh:  
pwd
```

9.1.2 Verzeichnis wechseln

```
# Funktioniert überall (Windows, Linux, macOS):  
cd Dokumente  
cd ..           # Ein Verzeichnis nach oben  
cd C:\Pfad\zum\Ordner  # Windows: Absoluter Pfad  
cd /home/user/ordner   # Linux/macOS: Absoluter Pfad
```

9.1.3 Dateien und Ordner auflisten

```
# Windows CMD:
dir

# PowerShell, bash, zsh:
ls

# In PowerShell funktioniert auch:
dir
```

9.1.4 Bildschirm löschen

```
# Windows CMD und PowerShell:
cls

# bash, zsh:
clear
```

9.2 B. Python-Skripte ausführen

9.2.1 Python-Skript erstellen und ausführen

1. Skript erstellen

Erstellen Sie eine Datei `hallo.py`:

```
print("Hallo Welt!")
```

2. Skript ausführen

Windows CMD:

```
# Python-Version prüfen
python --version
python3 --version

# Skript ausführen
python hallo.py
python3 hallo.py

# Mit vollständigem Pfad
python C:\Users\MeinName\Projekte\hallo.py

# Im Hintergrund ausführen (keine Ausgabe)
python hallo.py > nul
```

Windows PowerShell:

```
# Skript ausführen
python .\hallo.py
python3 .\hallo.py

# Mit Parameter
python .\mein_script.py argument1 argument2

# Ausgabe in Datei umleiten
python .\hallo.py > ausgabe.txt
```

Linux/macOS Terminal:

```
# Skript ausführen
python3 hallo.py

# Skript direkt ausführbar machen (Shebang)
# Erste Zeile in hallo.py: #!/usr/bin/env python3
chmod +x hallo.py
./hallo.py
```

9.2.2 Python Interactive Shell

```
# Python-Shell starten
python
python3

# Python-Shell beenden
exit()
Ctrl+Z (Windows) oder Ctrl+D (Linux/macOS)

# IPython starten (falls installiert)
ipython
```

9.2.3 Pip - Paketmanager

```
# Pip-Version prüfen
pip --version
pip3 --version

# Paket installieren
pip install requests
pip install pandas numpy

# Paket deinstallieren
pip uninstall requests

# Installierte Pakete auflisten
pip list
pip freeze

# Requirements installieren
```

(Fortsetzung auf der nächsten Seite)

(Fortsetzung der vorherigen Seite)

```
pip install -r requirements.txt

# Requirements-Datei erstellen
pip freeze > requirements.txt

# Paket upgraden
pip install --upgrade requests
```

9.2.4 Virtuelle Umgebungen

Erstellen und Aktivieren:

```
# Windows CMD:
python -m venv meinenv
meinenv\Scripts\activate

# Windows PowerShell:
python -m venv meinenv
meinenv\Scripts\Activate.ps1

# Linux/macOS:
python3 -m venv meinenv
source meinenv/bin/activate

# Deaktivieren (alle Systeme):
deactivate
```

9.2.5 Python-Skripte mit Argumenten

Skript (args.py):

```
import sys

print(f"Script-Name: {sys.argv[0]}")
print(f"Anzahl Argumente: {len(sys.argv) - 1}")

for i, arg in enumerate(sys.argv[1:], 1):
    print(f"Argument {i}: {arg}")
```

Ausführen:

```
python args.py eins zwei drei
# Ausgabe:
# Script-Name: args.py
# Anzahl Argumente: 3
# Argument 1: eins
# Argument 2: zwei
# Argument 3: drei
```

9.3 C. Offizielle Python-Dokumentation

9.3.1 Wichtigste Ressourcen

Deutsche Dokumentation:

- Hauptseite: <https://docs.python.org/de/3/>
- Tutorial: <https://docs.python.org/de/3/tutorial/>
- Standardbibliothek: <https://docs.python.org/de/3/library/>

Englische Dokumentation:

- Python.org: <https://www.python.org/>
- Vollständige Dokumentation: <https://docs.python.org/3/>
- PEP 8 Style Guide: <https://pep8.org/>

PyPI - Python Package Index:

- Pakete suchen und installieren: <https://pypi.org/>

9.4 D. Nützliche Tastenkombinationen

9.4.1 Python Interactive Shell

```
Ctrl+D (Linux/macOS) oder Ctrl+Z (Windows) - Shell beenden  
Ctrl+C - Aktuellen Befehl abbrechen  
Pfeil hoch/runter - Befehlshistorie durchsuchen  
Tab - Auto-Vervollständigung  
Ctrl+L - Bildschirm löschen
```

9.4.2 Windows CMD

```
Tab - Dateinamen-Vervollständigung  
F7 - Befehlshistorie anzeigen  
Pfeil hoch/runter - Vorherige Befehle  
Ctrl+C - Prozess abbrechen
```

9.4.3 Editoren

VS Code:

```
Ctrl+Space - Auto-Vervollständigung  
F5 - Debugger starten  
Ctrl+/ - Zeile kommentieren  
Shift+Alt+F - Code formatieren  
Ctrl+` - Terminal öffnen/schließen
```

IDLE (Python IDE):

```
F5 - Skript ausführen
Alt+P - Vorheriger Befehl
Alt+N - Nächster Befehl
Ctrl+C - Kopieren
Ctrl+V - Einfügen
```

9.5 E. Häufige Fehler und Lösungen

9.5.1 ImportError / ModuleNotFoundError

```
# Fehler:
# ModuleNotFoundError: No module named 'requests'

# Lösung:
# pip install requests
```

9.5.2 SyntaxError: invalid syntax

```
# Häufige Ursachen:
# - Fehlende Klammern: print "Hallo" # Falsch
# - Fehlende Doppelpunkte: if x > 5 # Falsch
# - Falsche Einrückung

# Korrekt:
print("Hallo")
if x > 5:
    print("Größer als 5")
```

9.5.3 IndentationError

```
# Fehler: Inkonsistente Einrückung (Tabs und Leerzeichen gemischt)
# Lösung: Nur Leerzeichen verwenden (4 pro Ebene)
```

9.5.4 NameError: name 'x' is not defined

```
# Fehler: Variable vor Zuweisung verwendet
# print(ergebnis) # NameError

# Lösung: Variable zuerst definieren
ergebnis = 42
print(ergebnis)
```

9.5.5 FileNotFoundError

```
# Fehler: Datei nicht gefunden
# with open("datei.txt") as f: # FileNotFoundError

# Lösungen:
# 1. Pfad prüfen
import os
print(os.getcwd()) # Aktuelles Verzeichnis

# 2. Absoluten Pfad verwenden
# with open("C:\\Users\\Name\\datei.txt") as f:

# 3. Existenz prüfen
from pathlib import Path
if Path("datei.txt").exists():
    with open("datei.txt") as f:
        print(f.read())
```

Weitere Hilfe:

- Python-FAQ: <https://docs.python.org/de/3/faq/>
- Häufige Fehler: <https://docs.python.org/de/3/faq/programming.html>